AMRITA VISHWA VIDYAPEETHAM

Lab Record: Experiments 1 to 6

Quantum Computing and Algorithms

SUBMITTED BY

Ritesh Roshan Sahoo

DL.AI.U4AID24036
riteshroshansahoo01@gmail.com

UNDER THE GUIDANCE OF

Dr. Mrittunjoy G. Majumdar
Lab Instructor: Sakshi

11 November 2026

github.com/riteshroshann/qca-labs

Contents

1 Experiment 1: Quantum State Evolution on the Bloch Sphere 1
- i. Aim 1
- ii. Setup 1
- iii. Task 1: Quantum Circuit Optimization Challenge 3
- iv. Task 2: Reverse Engineer the Unknown Circuit 6
- v. Task 3: Quantum Gate Discovery 11
- vi. Task 4: Bloch Sphere Navigation Challenge 17
- vii. Task 5: Beyond the Bloch Sphere 20
- viii. Conclusion 26

2 Experiment 2: Deutsch Algorithm 27
- i. Aim 27
- ii. Setup 27
- iii. Task 1: Basic Deutsch Algorithm 28
- iv. Task 2: Construct and Identify All Oracles 31
- v. Task 3: Phase Kickback Investigation 35
- vi. Task 4: Classical vs Quantum Query Advantage 39
- vii. Task 5: Noise and Oracle Equivalence 41
- viii. Conclusion 45

3 Experiment 3: Bernstein-Vazirani Algorithm 46
- i. Aim 46
- ii. Setup 46
- iii. Task 1: Basic Bernstein-Vazirani Implementation 47
- iv. Task 2: Hidden Oracle / Reverse Engineering 49
- v. Task 3: Classical vs Quantum Query Complexity 52
- vi. Task 4: Phase Kickback Investigation 54
- vii. Task 5: Noise, Scalability and Circuit Optimization 57
- viii. Conclusion 60

4 Experiment 4: Simon's Algorithm 61
- i. Aim 61
- ii. Setup 61
- iii. Task 1: Hidden Pattern Identification 63
- iv. Task 2: Black-Box Duplicate Detection 66
- v. Task 3: Reverse Engineering a Black-Box System 69
- vi. Task 4: Cryptographic Security Investigation 72
- vii. Task 5: Noise and Scalability 74
- viii. Conclusion 77

5 Experiment 5: Grover's Search Algorithm 78
- i. Aim 78
- ii. Setup 78
- iii. Task 1: Basic Grover Search 79

iv. Task 2: Oracle Design for a Real-World Search Problem 82
v. Task 3: Optimal Number of Grover Iterations 86
vi. Task 4: Multiple Target States 89
vii. Task 5: Noise, Search Accuracy and Scalability 92
viii. Conclusion 95

6 Experiment 6: Shor's Algorithm 96
i. Aim 96
ii. Setup 96
iii. Task 1: Classical Period Finding 97
iv. Task 2: Construction of Modular Multiplication Gates 99
v. Task 3: Quantum Circuit Construction and Simulation 101
vi. Task 4: Period Estimation and Factor Recovery 104
vii. Task 5: Classical and Quantum Comparison 106
viii. Task 6: Noise and Scalability Investigation 108
ix. Conclusion 111

CHAPTER 1

Experiment 1: Quantum State Evolution on the Bloch Sphere

Exploring Quantum State Evolution Using the Bloch Sphere in Qiskit

i. Aim

To prepare, visualise and optimise single-qubit states in Qiskit using the Bloch sphere, to reverse engineer circuits from observed outputs, to characterise the standard gates experimentally and to investigate where the Bloch sphere stops being a complete description.

ii. Setup

The notebook installs Qiskit and Qiskit Aer, then imports the simulator, visualisation and helper functions used by every task.

```
%pip install --quiet qiskit qiskit-aer matplotlib pylatexenc
```

```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

from qiskit import QuantumCircuit, transpile
from qiskit.quantum_info import Statevector, Operator, Pauli
from qiskit.visualization import plot_bloch_multivector,
plot_histogram
from qiskit.visualization.bloch import Bloch
from qiskit_aer import AerSimulator

simulator = AerSimulator()

# Display circuit using Matplotlib and pylatexenc
def show_circuit(qc, title):
    print("\n" + title)
```

```
    display(qc.draw(output="mpl"))

# Display statevector
def show_state(qc):
    print("Statevector:", np.round(
        Statevector.from_instruction(qc).data, 3
    ))

# Display measurement histogram
def show_histogram(qc, title):
    measured = qc.copy()
    measured.measure_all()
    result = simulator.run(
        transpile(measured, simulator), shots=1024
    ).result()
    counts = result.get_counts()
    print("Measurement counts:", counts)
    display(plot_histogram(counts, title=title))

# Bloch vector of a state as (x, y, z)
def bloch_xyz(state):
    return [state.expectation_value(Pauli(p)).real for p in "XYZ"]

# One Bloch sphere after every gate of a single-qubit circuit
def bloch_evolution(qc):
    steps = len(qc.data)
    fig = plt.figure(figsize=(3.2 * (steps + 1), 3.2))
    partial = QuantumCircuit(1)
    for k in range(steps + 1):
        if k > 0:
            partial.append(qc.data[k - 1])
        state = Statevector.from_instruction(partial)
        ax = fig.add_subplot(1, steps + 1, k + 1, projection="3d")
        sphere = Bloch(fig=fig, axes=ax)
        sphere.add_vectors([bloch_xyz(state)])
        if k == 0:
            name = "start"
        else:
            name = "after " + qc.data[k - 1].operation.name.upper()
        sphere.render(title=name)
    plt.close(fig)
    return fig
```

iii. Task 1: Quantum Circuit Optimization Challenge

Problem Statement

The target state is $|1\rangle$. Design at least three different circuits that prepare it from $|0\rangle$ using only the gates X, Y, Z, H, S, T (no rotation gates), verify each with the statevector, plot the Bloch sphere evolution, optimise each circuit with the Qiskit transpiler and compare depth, gate count, number of phase gates and the transpiled metrics.

Qiskit Code and Output

```
# Three circuits that prepare |1> from |0>
q1 = QuantumCircuit(1)
q1.x(0)

q2 = QuantumCircuit(1)
q2.h(0)
q2.z(0)
q2.h(0)

q3 = QuantumCircuit(1)
q3.h(0)
q3.s(0)
q3.s(0)
q3.h(0)

phase_gates = {"z", "s", "t", "sdg", "tdg"}

for i, qc in enumerate([q1, q2, q3], 1):
    show_circuit(qc, f"Task 1 - Circuit {i}")
    show_state(qc)
    display(bloch_evolution(qc))

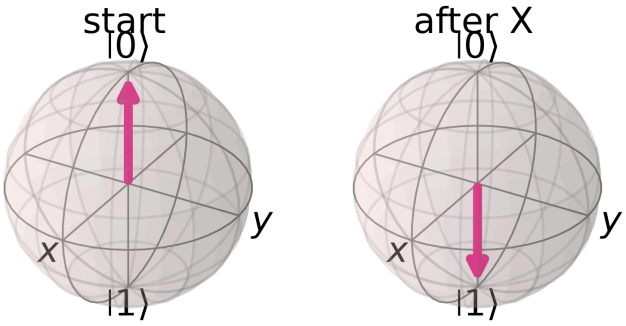
    optimized = transpile(
        qc, basis_gates=["u", "cx"], optimization_level=3
    )
    ops = qc.count_ops()
    print("Depth:", qc.depth())
    print("Gate counts:", dict(ops))
    print("Phase gates:",
          sum(ops[g] for g in ops if g in phase_gates))
    print("Optimized depth:", optimized.depth())
    print("Optimized gate counts:", dict(optimized.count_ops()))
```

Output

```
Task 1 - Circuit 1
```

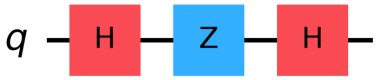



```
Statevector: [0.+0.j 1.+0.j]
```

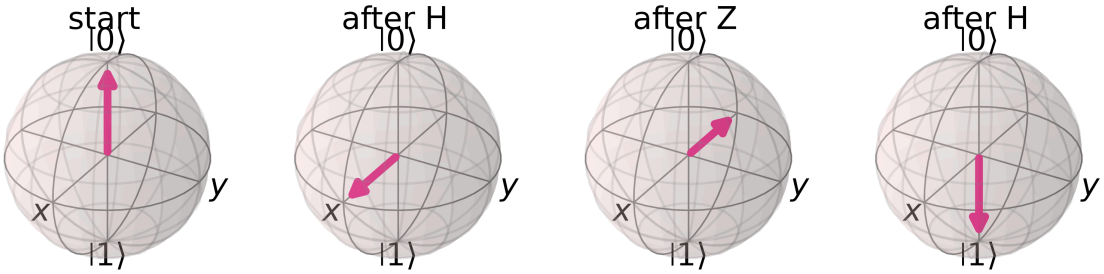



```
Depth: 1
Gate counts: {'x': 1}
Phase gates: 0
Optimized depth: 1
Optimized gate counts: {'u': 1}

Task 1 - Circuit 2
```



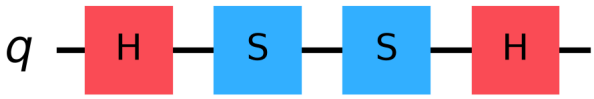

```
Statevector: [0.+0.j 1.+0.j]
```



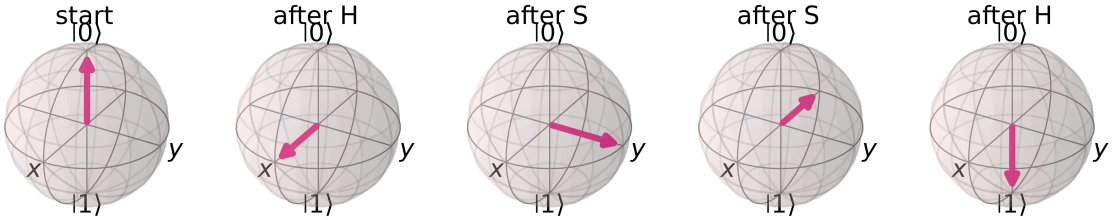

```
Depth: 3
Gate counts: {'h': 2, 'z': 1}
Phase gates: 1
```

```
Optimized depth: 1
Optimized gate counts: {'u': 1}

Task 1 - Circuit 3
```




```
Statevector: [0.+0.j 1.+0.j]
```




```
Depth: 4
Gate counts: {'h': 2, 's': 2}
Phase gates: 2
Optimized depth: 1
Optimized gate counts: {'u': 1}
```

Observation

Comparison of the three preparations of the target state

Circuit	Gates	Depth	Phase gates	After transpile (u, cx)
1: X	1	1	0	1 gate, depth 1
2: H Z H	3	3	1	1 gate, depth 1
3: H S S H	4	4	2	1 gate, depth 1

All three circuits give the statevector $[0, 1]$ and the same Bloch vector $(0, 0, -1)$, so they prepare the same state. **Circuit 1 is optimal**: one gate, depth 1 and no phase gates. The transpiler recognises that $HZH = X$ and $SS = Z$ and **collapses circuits 2 and 3 to a single u gate**, which confirms that the extra gates were pure overhead.

iv. Task 2: Reverse Engineer the Unknown Circuit

Problem Statement

A hidden single-qubit circuit maps $|0\rangle$ to $|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$ with a 512/512 histogram and Bloch vector $(1, 0, 0)$. Reconstruct at least two different circuits that produce this output, verify the statevector, histogram and Bloch sphere, compute the unitary of each and decide whether the circuits are the same operation. Then build two circuits with **different** unitaries that still give $|+\rangle$ for the input $|0\rangle$, and transpile every circuit to see what Qiskit can simplify.

Qiskit Code and Output

```
# Circuit A: H |0> = |+>
c1 = QuantumCircuit(1)
c1.h(0)

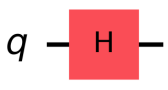
# Circuit B: H followed by S and S-dagger
c2 = QuantumCircuit(1)
c2.h(0)
c2.s(0)
c2.sdg(0)

for name, qc in [("Circuit A", c1), ("Circuit B", c2)]:
    show_circuit(qc, "Task 2 - " + name)
    show_state(qc)
    print("Unitary matrix:\n", np.round(Operator(qc).data, 3))
    show_histogram(qc, name)
    display(plot_bloch_multivector(
        Statevector.from_instruction(qc)
    ))

print("Same unitary:", Operator(c1).equiv(Operator(c2)))
```

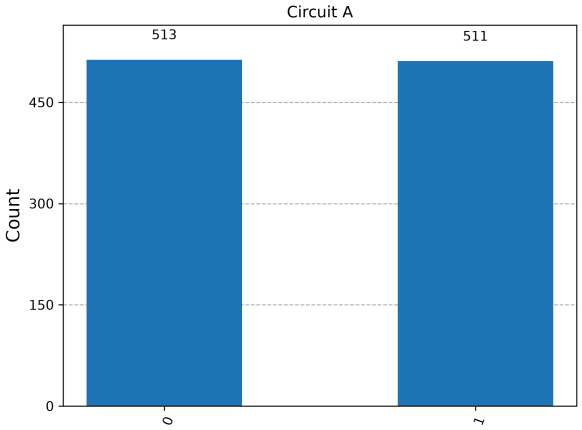
Output

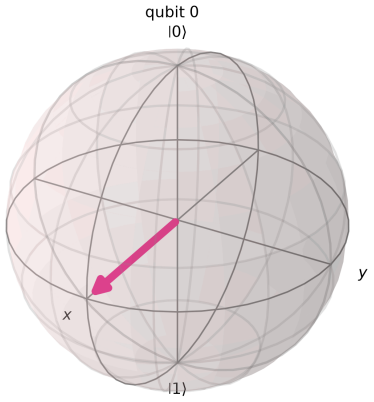
```
Task 2 - Circuit A
```



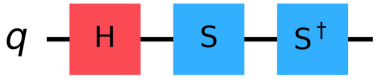

```
Statevector: [0.707+0.j 0.707+0.j]
Unitary matrix:
 [[ 0.707+0.j  0.707+0.j]
```

```
 [ 0.707+0.j -0.707+0.j]]
Measurement counts: {'0': 513, '1': 511}
```

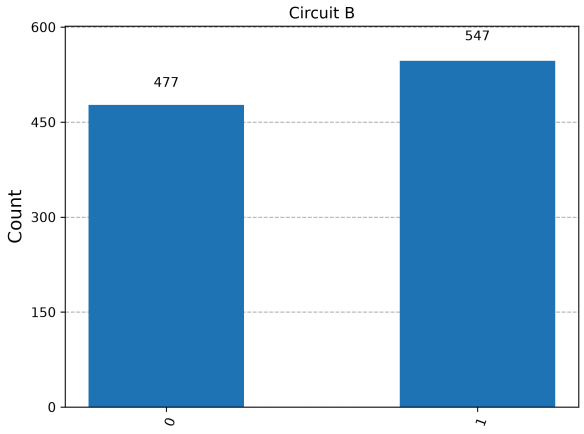


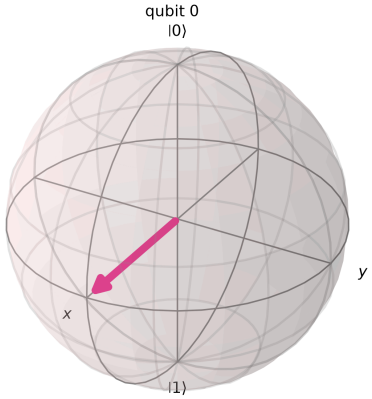



```
Task 2 - Circuit B
```




```
Statevector: [0.707+0.j 0.707+0.j]
Unitary matrix:
 [[ 0.707+0.j  0.707+0.j]
 [ 0.707+0.j -0.707+0.j]]
Measurement counts: {'1': 547, '0': 477}
```






```
Same unitary: True
```

```
# Extension: same output for |0>, different unitary
e1 = QuantumCircuit(1)
e1.h(0)

e2 = QuantumCircuit(1)
e2.s(0)
e2.h(0)

print("U_C1 =\n", np.round(Operator(e1).data, 3))
print("U_C2 =\n", np.round(Operator(e2).data, 3))
print("Same unitary:", Operator(e1).equiv(Operator(e2)))

for label in ["0", "1", "+"]:
    out1 = Statevector.from_label(label).evolve(e1)
    out2 = Statevector.from_label(label).evolve(e2)
    print(f"\nInput |{label}>")
    print("  C1 output:", np.round(out1.data, 3))
    print("  C2 output:", np.round(out2.data, 3))
    print("  Same state:", out1.equiv(out2))
```

Output

```
U_C1 =
 [[ 0.707+0.j  0.707+0.j]
 [ 0.707+0.j -0.707+0.j]]
U_C2 =
 [[0.707+0.j    0.   +0.707j]
 [0.707+0.j    0.   -0.707j]]
Same unitary: False

Input |0>
  C1 output: [0.707+0.j 0.707+0.j]
  C2 output: [0.707+0.j 0.707+0.j]
  Same state: True

Input |1>
  C1 output: [ 0.707+0.j -0.707+0.j]
  C2 output: [0.+0.707j 0.-0.707j]
  Same state: True

Input |+>
  C1 output: [1.+0.j 0.+0.j]
  C2 output: [0.5+0.5j 0.5-0.5j]
  Same state: False
```

```
# Circuit optimisation with the transpiler
circuits = [("Circuit A", c1), ("Circuit B", c2), ("C2 = S H", e2)]
for name, qc in circuits:
    optimized = transpile(
        qc, basis_gates=["u", "cx"], optimization_level=3
    )
    print(name)
    print("  original : depth", qc.depth(),
          "gates", dict(qc.count_ops()))
    print("  optimized: depth", optimized.depth(),
          "gates", dict(optimized.count_ops()))
```

Output

```
Circuit A
  original : depth 1 gates {'h': 1}
  optimized: depth 1 gates {'u': 1}
Circuit B
  original : depth 3 gates {'h': 1, 's': 1, 'sdg': 1}
  optimized: depth 1 gates {'u': 1}
C2 = S H
  original : depth 2 gates {'s': 1, 'h': 1}
  optimized: depth 1 gates {'u': 1}
```

Observation

Circuits A and B implement exactly the same unitary, the Hadamard matrix, because $SS^{\dagger} = I$. Both give the statevector $[0.707, 0.707]$, a histogram close to 512/512 and the Bloch vector $(1, 0, 0)$.

In the extension, $C_1 = H$ and $C_2 = S$ followed by H have **different unitaries** but both map $|0\rangle$ to $|+\rangle$, because $S|0\rangle = |0\rangle$. For the input $|1\rangle$ the outputs differ only by a global phase i, so they are still the same physical state, but for the input $|+\rangle$ the outputs are different states. **Equal output for one input does not mean equal operation**; a unitary is only fixed by its action on a full basis.

The transpiler reduces Circuit B from **3 gates to 1**, since the cancelling pair is removed, while Circuit A is already minimal.

v. Task 3: Quantum Gate Discovery

Problem Statement

Treat X, Y, Z, H, S, T as black boxes. Using only experiments, find the rotation axis and angle of each gate, its effect on measurement probabilities, the phase it introduces, whether it is self-inverse and whether it equals a sequence of other gates.

Experimental Procedure

- **Axis and angle**: apply the gate to $|0\rangle$, $|1\rangle$ and $|+\rangle$ and plot the three output Bloch vectors on one sphere. The vector that does not move lies on the rotation axis; the angle is read from how far the other vectors turn.
- **Probabilities and phase**: print the output statevector and measurement probabilities for each input.
- **Self-inverse**: apply the gate twice and compare with the input state.
- **Equivalences**: compare the unitary of candidate gate sequences with `Operator.equiv`.

Qiskit Code and Output

```
gates = ["X", "Y", "Z", "H", "S", "T"]
inputs = ["0", "1", "+"]

def apply_gate(gate, input_state):
    qc = QuantumCircuit(1)
    if input_state == "1":
        qc.x(0)
    elif input_state == "+":
        qc.h(0)
    getattr(qc, gate.lower())(0)
    return Statevector.from_instruction(qc)

for gate in gates:
    qc = QuantumCircuit(1)
    getattr(qc, gate.lower())(0)
    print(f"\n===== {gate} GATE =====")
    print(qc.draw())
    print("Unitary matrix:\n", np.round(Operator(qc).data, 3))

    sphere = Bloch()
    sphere.vector_color = ["r", "g", "b"]
    for state in inputs:
        output = apply_gate(gate, state)
        print(f"|{state}> -> {np.round(output.data, 3)}",
              "Probabilities:", np.round(output.probabilities(), 3))
        sphere.add_vectors([bloch_xyz(output)])
```

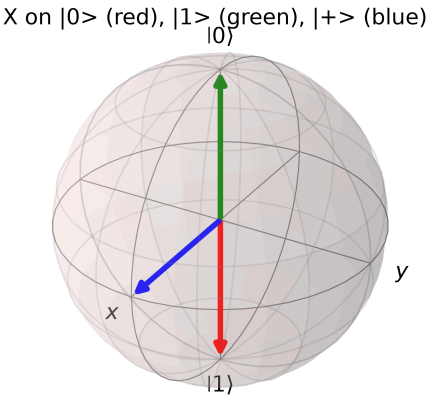
```
    sphere.render(
        title=f"{gate} on |0> (red), |1> (green), |+> (blue)"
    )
    display(sphere.fig)
    plt.close(sphere.fig)
```

Output

```
===== X GATE =====

q: ┤ X ├

Unitary matrix:
 [[0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j]]
|0> -> [0.+0.j 1.+0.j] Probabilities: [0. 1.]
|1> -> [1.+0.j 0.+0.j] Probabilities: [1. 0.]
|+> -> [0.707+0.j 0.707+0.j] Probabilities: [0.5 0.5]
```

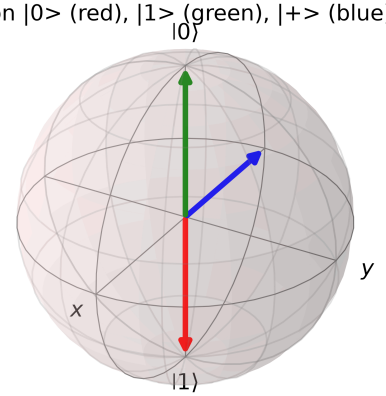



```
===== Y GATE =====

q: ┤ Y ├

Unitary matrix:
 [[0.+0.j 0.-1.j]
 [0.+1.j 0.+0.j]]
|0> -> [0.+0.j 0.+1.j] Probabilities: [0. 1.]
|1> -> [0.-1.j 0.+0.j] Probabilities: [1. 0.]
|+> -> [0.-0.707j 0.+0.707j] Probabilities: [0.5 0.5]
```

Y on |0> (red), |1> (green), |+> (blue)

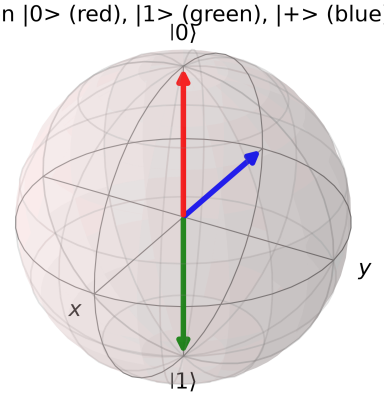



```
===== Z GATE =====

q: ┤ Z ├

Unitary matrix:
 [[ 1.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j]]
|0> -> [1.+0.j 0.+0.j] Probabilities: [1. 0.]
|1> -> [ 0.+0.j -1.+0.j] Probabilities: [0. 1.]
|+> -> [ 0.707+0.j -0.707+0.j] Probabilities: [0.5 0.5]
```

Z on |0> (red), |1> (green), |+> (blue)

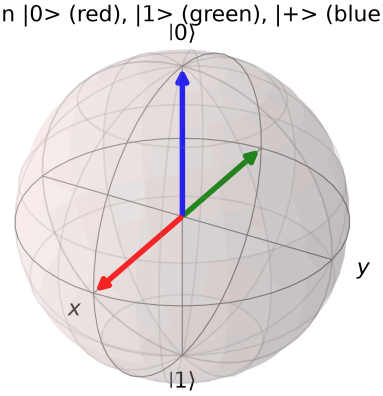



```
===== H GATE =====

q: ┤ H ├

Unitary matrix:
 [[ 0.707+0.j  0.707+0.j]
 [ 0.707+0.j -0.707+0.j]]
|0> -> [0.707+0.j 0.707+0.j] Probabilities: [0.5 0.5]
|1> -> [ 0.707+0.j -0.707+0.j] Probabilities: [0.5 0.5]
|+> -> [1.+0.j 0.+0.j] Probabilities: [1. 0.]
```

H on |0> (red), |1> (green), |+> (blue)

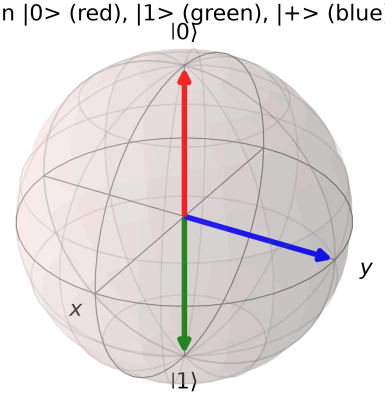



```
===== S GATE =====

q: ┤ S ├

Unitary matrix:
 [[1.+0.j 0.+0.j]
 [0.+0.j 0.+1.j]]
|0> -> [1.+0.j 0.+0.j] Probabilities: [1. 0.]
|1> -> [0.+0.j 0.+1.j] Probabilities: [0. 1.]
|+> -> [0.707+0.j    0.   +0.707j] Probabilities: [0.5 0.5]
```

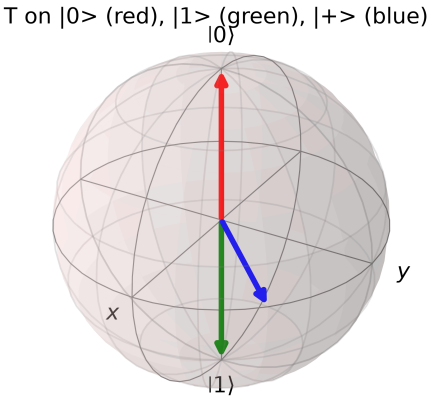
S on |0> (red), |1> (green), |+> (blue)




```
===== T GATE =====

q: ┤ T ├

Unitary matrix:
 [[1.   +0.j    0.   +0.j   ]
 [0.   +0.j    0.707+0.707j]]
|0> -> [1.+0.j 0.+0.j] Probabilities: [1. 0.]
|1> -> [0.   +0.j    0.707+0.707j] Probabilities: [0. 1.]
|+> -> [0.707+0.j  0.5  +0.5j] Probabilities: [0.5 0.5]
```




```
# Self-inverse test: apply each gate twice
def apply_twice(gate, input_state):
    qc = QuantumCircuit(1)
    for _ in range(2):
        getattr(qc, gate.lower())(0)
    return Statevector.from_label(input_state).evolve(qc)

for gate in gates:
    same = True
    for state in inputs:
        original = Statevector.from_label(state)
        output = apply_twice(gate, state)
        if not np.allclose(original.data, output.data, atol=1e-8):
            same = False
    print(f"{gate} Self-Inverse: {same}")
```

Output

```
X Self-Inverse: True
Y Self-Inverse: True
Z Self-Inverse: True
H Self-Inverse: True
S Self-Inverse: False
T Self-Inverse: False
```

```
# Equivalent circuits, compared through their unitaries
def unitary(*names):
    qc = QuantumCircuit(1)
    for name in names:
        getattr(qc, name)(0)
    return Operator(qc)

checks = {
    "S S = Z": (unitary("s", "s"), unitary("z")),
    "T T = S": (unitary("t", "t"), unitary("s")),
    "H Z H = X": (unitary("h", "z", "h"), unitary("x")),
    "H X H = Z": (unitary("h", "x", "h"), unitary("z")),
    "X Z = Y (up to phase)": (unitary("x", "z"), unitary("y")),
}
```

```
for name, (a, b) in checks.items():
    print(f"{name}: {a.equiv(b)}")
```

Output

```
S S = Z: True
T T = S: True
H Z H = X: True
H X H = Z: True
X Z = Y (up to phase): True
```

Observation

Experimental investigation of single-qubit gates

Gate	Rotation axis	Angle	Self-inverse	Phase change	Equivalent circuit
X	x	π	Yes	none on $\|+\rangle$	H Z H
Y	y	π	Yes	$\pm i$ on $\|0\rangle, \|1\rangle$	X Z up to phase
Z	z	π	Yes	-1 on $\|1\rangle$	S S, H X H
H	(x+z)/$\sqrt{2}$	π	Yes	none	self-inverse
S	z	$\frac{\pi}{2}$	No	i on $\|1\rangle$	T T
T	z	$\frac{\pi}{4}$	No	$e^{i\frac{\pi}{4}}$ on $\|1\rangle$	square root of S

The Bloch plots show which input vector stays fixed: $|+\rangle$ for X, $|0\rangle$ and $|1\rangle$ for Z, S and T. **Z, S and T all rotate about the z axis** and never change the measurement probabilities of $|0\rangle$ or $|1\rangle$; they differ only in the phase on $|1\rangle$. **X, Y, Z and H are self-inverse** (rotation by π) while **S and T are not**, and the equivalences $SS = Z$, $TT = S$ and $HZH = X$ were all confirmed by `Operator.equiv`.

vi. Task 4: Bloch Sphere Navigation Challenge

Problem Statement

Design one circuit that moves the qubit through the five states $|0\rangle \to |1\rangle \to |-\rangle \to |-i\rangle \to |+i\rangle$ in order. Record the Bloch sphere and statevector after every gate, explain each transition, minimise the number of gates, depth and phase gates, compare with another valid solution and decide experimentally whether more than one optimal navigation circuit exists.

Qiskit Code and Output

```
q4 = QuantumCircuit(1)
sequence = ["x", "h", "s", "z"]

for i, gate in enumerate(sequence, 1):
    getattr(q4, gate)(0)
    sv = Statevector.from_instruction(q4)
    print(f"After gate {i} ({gate.upper()}):", np.round(sv.data, 3),
          " Bloch:", np.round(bloch_xyz(sv), 3))

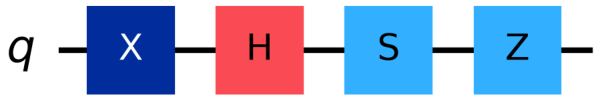
show_circuit(q4, "Task 4 - Complete Navigation Circuit")
display(bloch_evolution(q4))

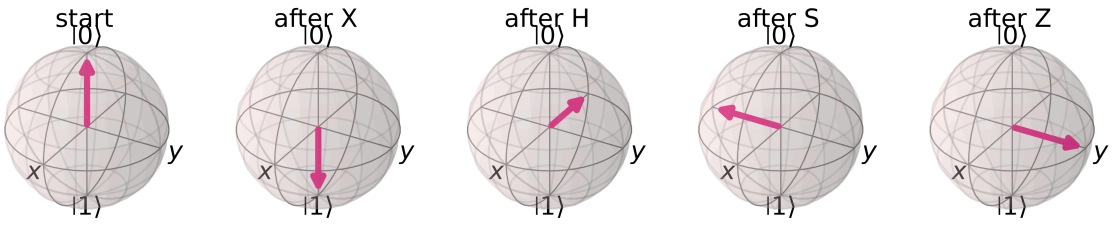
print("Depth:", q4.depth())
print("Gate counts:", dict(q4.count_ops()))
```

Output

```
After gate 1 (X): [0.+0.j 1.+0.j]  Bloch: [ 0.  0. -1.]
After gate 2 (H): [ 0.707+0.j -0.707+0.j]  Bloch: [-1.  0.  0.]
After gate 3 (S): [0.707+0.j    0.   -0.707j]  Bloch: [ 0. -1.  0.]
After gate 4 (Z): [0.707+0.j    0.   +0.707j]  Bloch: [0. 1. 0.]

Task 4 - Complete Navigation Circuit
```





```
Depth: 4
Gate counts: {'x': 1, 'h': 1, 's': 1, 'z': 1}
```

```
# Extension: search every 4-gate circuit visiting the same states
from itertools import product

allowed = ["x", "y", "z", "h", "s", "t"]
targets = [Statevector.from_label(l) for l in ["1", "-", "l", "r"]]

def visits_all(seq):
    qc = QuantumCircuit(1)
    for gate, target in zip(seq, targets):
        getattr(qc, gate)(0)
        if not Statevector.from_instruction(qc).equiv(target):
            return False
    return True

solutions = [s for s in product(allowed, repeat=4) if visits_all(s)]
print("Circuits visiting |1>, |->, |-i>, |+i> in order:")
for seq in solutions:
    phase = sum(g in ("z", "s", "t") for g in seq)
    print("  ", " -> ".join(g.upper() for g in seq),
          "| phase gates:", phase)
```

Output

```
Circuits visiting |1>, |->, |-i>, |+i> in order:
   X -> H -> S -> X | phase gates: 1
   X -> H -> S -> Z | phase gates: 2
   X -> H -> S -> H | phase gates: 1
   Y -> H -> S -> X | phase gates: 1
   Y -> H -> S -> Z | phase gates: 2
   Y -> H -> S -> H | phase gates: 1
```

Observation

Navigation sequence X, H, S, Z

Step	Gate	State	Why the transition happens
1	X	$\|1\rangle$	X flips $\|0\rangle$ to $\|1\rangle$ (rotation by π about x)
2	H	$\|-\rangle$	H maps $\|1\rangle$ to $(\|0\rangle - \|1\rangle)/\sqrt{2}$
3	S	$\|-i\rangle$	S multiplies the $\|1\rangle$ amplitude by i: $-1 \to -i$
4	Z	$\|+i\rangle$	Z multiplies the $\|1\rangle$ amplitude by -1: $-i \to +i$

Every intermediate statevector and Bloch vector matched the target, so the circuit **navigates all five states with 4 gates and depth 4**. The exhaustive search found **exactly six valid 4-gate circuits**: the first move can be X or Y, the middle moves must be H then S, and the last move can be X, Z or H. Since each state change needs at least one gate, four gates is the minimum, so **more than one optimal navigation circuit exists**. Among them X H S

X and X H S H use only **one phase gate** against two in X H S Z, so they are the better choice when phase gates are counted.

vii. Task 5: Beyond the Bloch Sphere

Problem Statement

Investigate whether the Bloch sphere still describes multi-qubit states. For the product state $|+\rangle\ |1\rangle$, the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$, the GHZ state $(|000\rangle + |111\rangle)/\sqrt{2}$ and the W state $(|001\rangle + |010\rangle + |100\rangle)/\sqrt{3}$: build the circuit, plot the individual Bloch spheres, plot the histogram, display the statevector and decide whether the Bloch spheres describe the state completely. Finally propose a criterion for when the Bloch sphere is sufficient.

Qiskit Code and Output

```
# Product state |+>|1>
product = QuantumCircuit(2)
product.h(0)
product.x(1)

# Bell state (|00> + |11>) / sqrt(2)
bell = QuantumCircuit(2)
bell.h(0)
bell.cx(0, 1)

# GHZ state (|000> + |111>) / sqrt(2)
ghz = QuantumCircuit(3)
ghz.h(0)
ghz.cx(0, 1)
ghz.cx(0, 2)

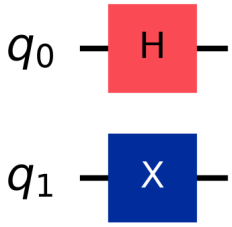
# W state (|001> + |010> + |100>) / sqrt(3)
w = QuantumCircuit(3)
state = np.zeros(8, dtype=complex)
state[1] = 1 / np.sqrt(3)
state[2] = 1 / np.sqrt(3)
state[4] = 1 / np.sqrt(3)
w.initialize(state, [0, 1, 2])

states = [
    ("Product State", product),
    ("Bell State", bell),
    ("GHZ State", ghz),
    ("W State", w),
]

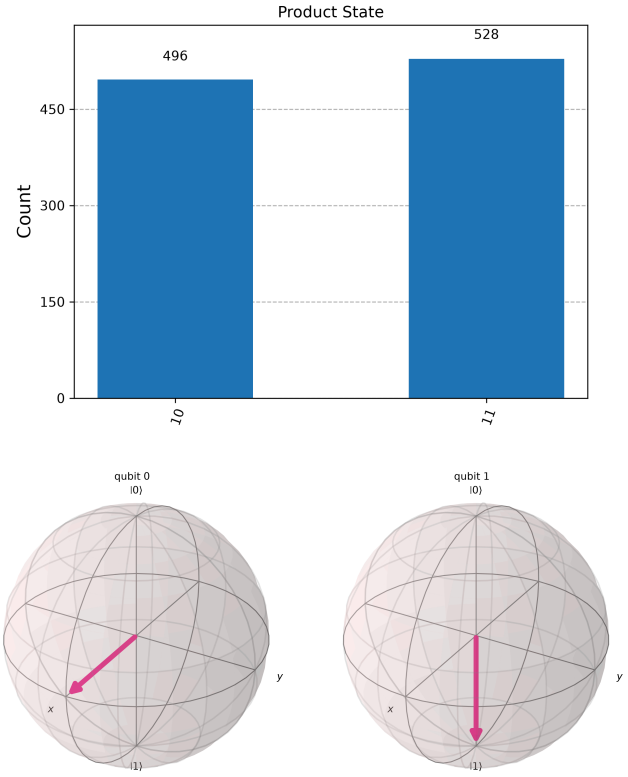
for name, qc in states:
    show_circuit(qc, "Task 5 - " + name)
    show_state(qc)
    show_histogram(qc, name)
    display(plot_bloch_multivector(
        Statevector.from_instruction(qc)
    ))
```

Output

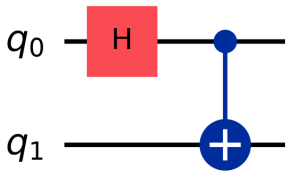
```
Task 5 - Product State
```



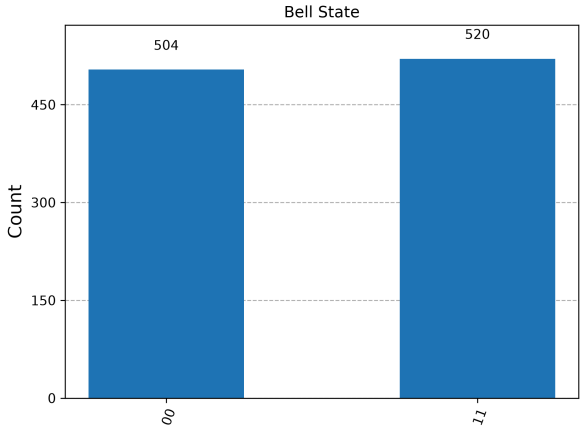

```
Statevector: [0.   +0.j 0.   +0.j 0.707+0.j 0.707+0.j]
Measurement counts: {'10': 496, '11': 528}
```

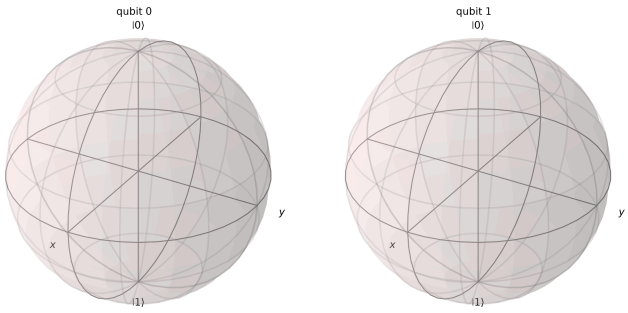



```
Task 5 - Bell State
```

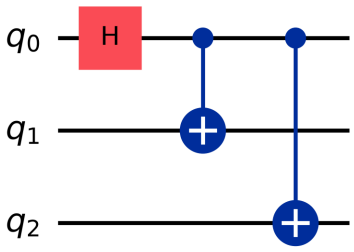



```
Statevector: [0.707+0.j 0.   +0.j 0.   +0.j 0.707+0.j]
Measurement counts: {'00': 504, '11': 520}
```

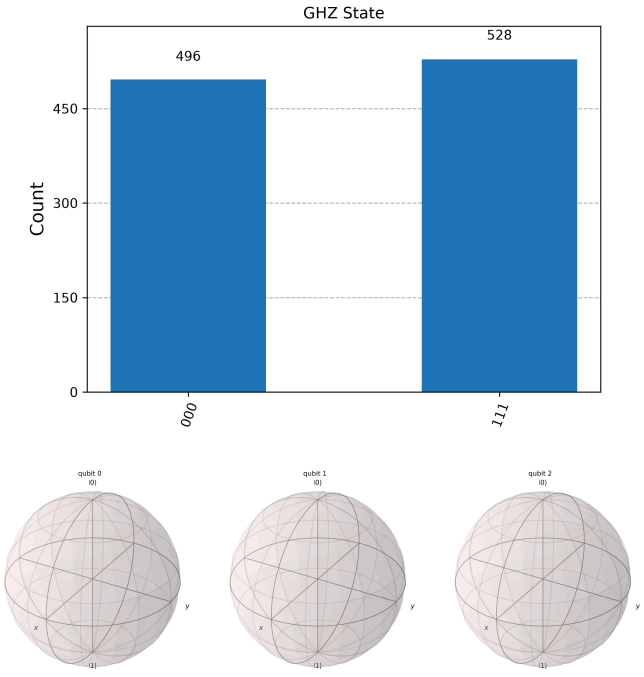




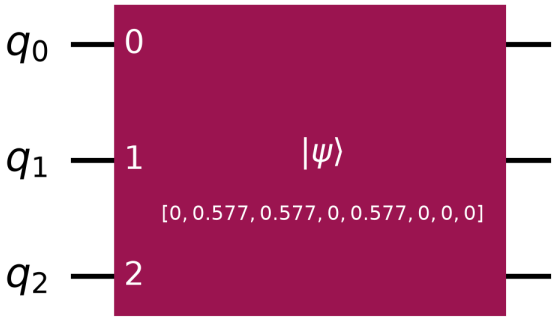

```
Task 5 - GHZ State
```



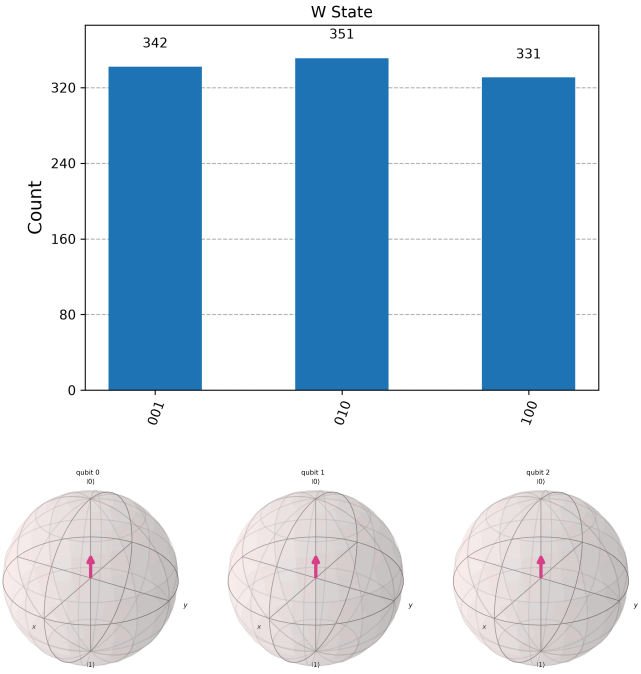

```
Statevector: [0.707+0.j 0.   +0.j 0.   +0.j 0.   +0.j 0.   +0.j 0.
+0.j 0.   +0.j
 0.707+0.j]
Measurement counts: {'000': 496, '111': 528}
```




```
Task 5 - W State
```




```
Statevector: [0.   +0.j 0.577+0.j 0.577+0.j 0.   +0.j 0.577+0.j 0.
+0.j 0.   +0.j
 0.   +0.j]
Measurement counts: {'010': 351, '001': 342, '100': 331}
```



Observation

Entanglement investigation

State	Histogram	Single-qubit Bloch vectors	Bloch sphere sufficient?
$\|+\rangle\ \|1\rangle$	10 and 11, about 50/50	$(1,0,0)$ and $(0,0,-1)$, full length	Yes

State	Histogram	Single-qubit Bloch vectors	Bloch sphere sufficient?
Bell	00 and 11, about 50/50	both at the origin	No
GHZ	000 and 111, about 50/50	all three at the origin	No
W	001, 010, 100, about 1/3 each	all three $\left(0, 0, \frac{1}{3}\right)$, shortened	No

For the product state each qubit has a **unit-length Bloch vector** and the two spheres reproduce the full statevector. For the Bell, GHZ and W states the **individual Bloch vectors shrink or vanish**: the reduced state of each qubit is mixed, so the spheres cannot show the correlations seen in the histogram (00 and 11 only, never 01 or 10).

Criterion: the Bloch sphere is a complete description only when every single-qubit Bloch vector has length 1, which happens exactly when the state is a product of pure single-qubit states. A vector of length less than 1 signals entanglement, and then the global statevector (or density matrix) is required.

viii. Conclusion

The same single-qubit state can be reached by many gate sequences, and the transpiler reliably finds the shortest one; a shared output state does not imply a shared unitary. The gates X, Y, Z, H, S and T were classified experimentally by axis, angle, phase and self-inverse behaviour. Finally, the Bloch sphere describes product states completely but fails for entangled states, where the single-qubit vectors shrink toward the origin.

CHAPTER 2

Experiment 2: Deutsch Algorithm

Implementation and Investigation of the Deutsch Algorithm Using Qiskit

i. Aim

To implement the Deutsch algorithm using Qiskit and experimentally investigate quantum superposition, oracle operations, phase kickback, interference and quantum query advantage.

ii. Setup

The notebook installs Qiskit and Qiskit Aer, then imports the simulator, visualisation and helper functions used by every task.

```
%pip install --quiet qiskit qiskit-aer matplotlib pylatexenc
```

```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

from qiskit import QuantumCircuit, transpile
from qiskit.quantum_info import Statevector, Operator
from qiskit.visualization import plot_histogram,
plot_bloch_multivector
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error

backend = AerSimulator()
```

iii. Task 1: Basic Deutsch Algorithm

Problem Statement

Construct the two-qubit Deutsch circuit and determine whether a given oracle $U_f|x, y\rangle = |x, y \oplus f(x)\rangle$ represents a constant or a balanced function. Measuring 0 on the input qubit means constant, measuring 1 means balanced. Run it for the constant oracle $f(x) = 0$ and the balanced oracle $f(x) = x$.

Qiskit Code and Output

```
def deutsch_algorithm(oracle):
    qc = QuantumCircuit(2, 1)
    # Prepare |0>|1>
    qc.x(1)
    # Create superposition
    qc.h(0)
    qc.h(1)
    # Oracle
    oracle(qc)
    # Interference
    qc.h(0)
    # Measure input qubit
    qc.measure(0, 0)
    return qc

# Constant Oracle: f(x) = 0
def constant_oracle(qc):
    pass

# Balanced Oracle: f(x) = x
def balanced_oracle(qc):
    qc.cx(0, 1)

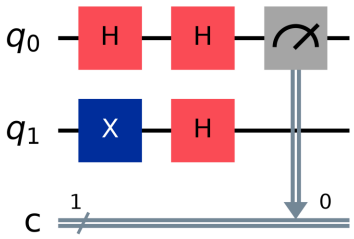
for name, oracle in [("Constant oracle f(x) = 0", constant_oracle),
                     ("Balanced oracle f(x) = x", balanced_oracle)]:
    qc = deutsch_algorithm(oracle)
    print("\n" + name)
    display(qc.draw("mpl"))

    compiled = transpile(qc, backend)
    result = backend.run(compiled, shots=1024).result()
    counts = result.get_counts()
    print("Measurement Results:", counts)
    display(plot_histogram(counts, title=name))
```

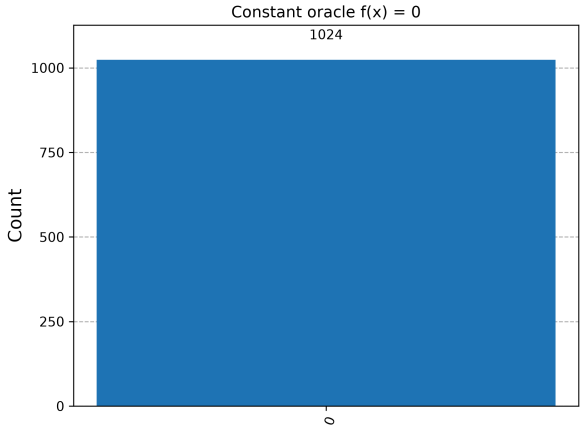
Output

```

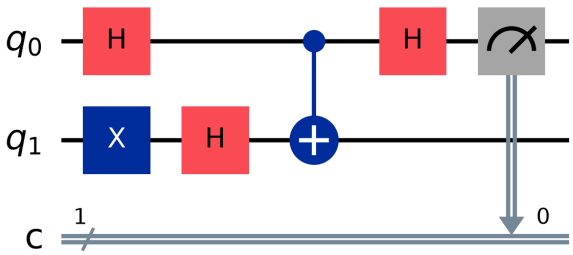
Constant oracle f(x) = 0
```



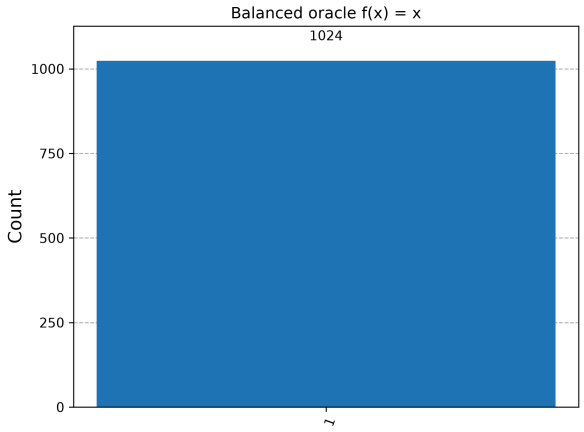

```
Measurement Results: {'0': 1024}
```




```
Balanced oracle f(x) = x
```




```
Measurement Results: {'1': 1024}
```



Observation

The constant oracle gives **1024 counts of 0** and the balanced oracle gives **1024 counts of 1**, exactly as predicted. The input qubit alone, after a single oracle call, tells whether f is constant or balanced.

iv. Task 2: Construct and Identify All Oracles

Problem Statement

Construct quantum oracles for all four Boolean functions $f_1(x) = 0$, $f_2(x) = 1$, $f_3(x) = x$ and $f_4(x) = x \oplus 1$, run Deutsch's algorithm on each and classify the function experimentally.

Qiskit Code and Output

```
# Oracle 1: f(x) = 0
def oracle_0(qc):
    pass

# Oracle 2: f(x) = 1
def oracle_1(qc):
    qc.x(1)

# Oracle 3: f(x) = x
def oracle_x(qc):
    qc.cx(0, 1)

# Oracle 4: f(x) = x XOR 1
def oracle_xor1(qc):
    qc.x(1)
    qc.cx(0, 1)

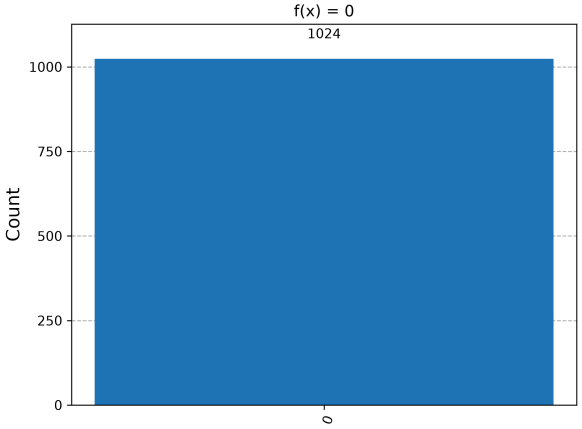
def deutsch(oracle):
    qc = QuantumCircuit(2, 1)
    qc.x(1)
    qc.h(0)
    qc.h(1)
    oracle(qc)
    qc.h(0)
    qc.measure(0, 0)
    return qc

oracles = {
    "f(x) = 0": oracle_0,
    "f(x) = 1": oracle_1,
    "f(x) = x": oracle_x,
    "f(x) = x XOR 1": oracle_xor1,
}

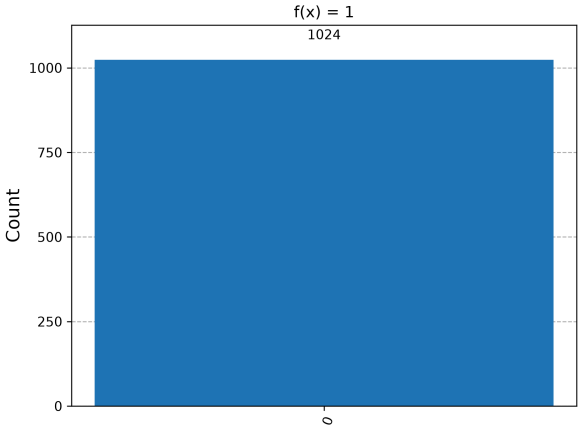
for name, oracle in oracles.items():
    qc = deutsch(oracle)
    result = backend.run(
        transpile(qc, backend), shots=1024
    ).result()
    counts = result.get_counts()
    kind = "Constant" if "0" in counts else "Balanced"
    print("\n", name, "->", counts, "->", kind)
    print(qc.draw())
    display(plot_histogram(counts, title=name))
```

Output

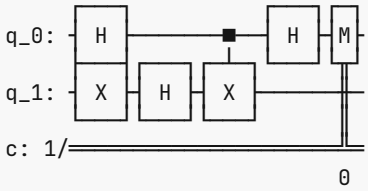
```
f(x) = 0 -> {'0': 1024} -> Constant
q_0: ┤ H ├┤ H ├┤ M ├
q_1: ┤ X ├┤ H ├─────
c: 1/═══════════╩═══
                0
```

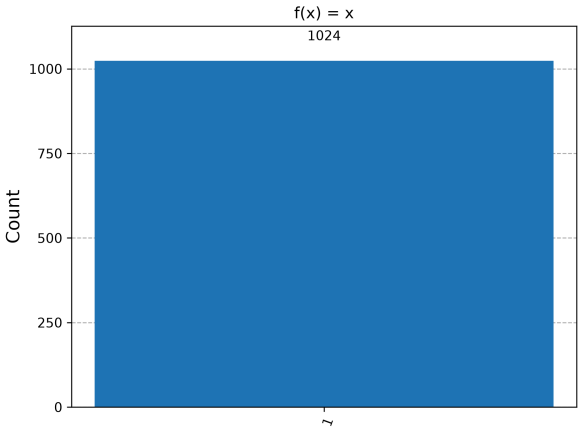



```
f(x) = 1 -> {'0': 1024} -> Constant
q_0: ┤ H ├┤ H ├──────┤ M ├
q_1: ┤ X ├┤ H ├┤ X ├─────
c: 1/═══════════╩═══
                0
```

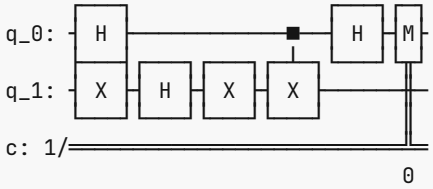


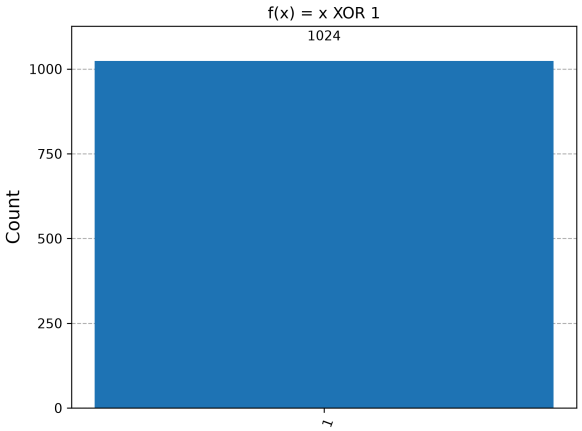
```
f(x) = x -> {'1': 1024} -> Balanced
```






```
f(x) = x XOR 1 -> {'1': 1024} -> Balanced
```





Observation

Classification of the four Boolean functions

Function	Expected type	Measured result	Experimental type
$f(x) = 0$	Constant	0 (1024 shots)	Constant
$f(x) = 1$	Constant	0 (1024 shots)	Constant
$f(x) = x$	Balanced	1 (1024 shots)	Balanced
$f(x) = x \oplus 1$	Balanced	1 (1024 shots)	Balanced

All four functions were classified correctly with one oracle query each. The X gate in oracles 2 and 4 only adds a global phase when the ancilla is $|-\rangle$, which is why $f = 1$ behaves like $f = 0$ and $f = x \oplus 1$ behaves like $f = x$.

v. Task 3: Phase Kickback Investigation

Problem Statement

The standard algorithm prepares the ancilla in $|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$. Prepare the ancilla instead as $|0\rangle$, $|1\rangle$, $|+\rangle$ and $|-\rangle$, apply the same balanced oracle $f(x) = x$ to the input $|+\rangle$, and investigate how the statevector and the ancilla change, whether information about $f(x)$ appears as a phase, and why $|-\rangle$ is required.

Qiskit Code and Output

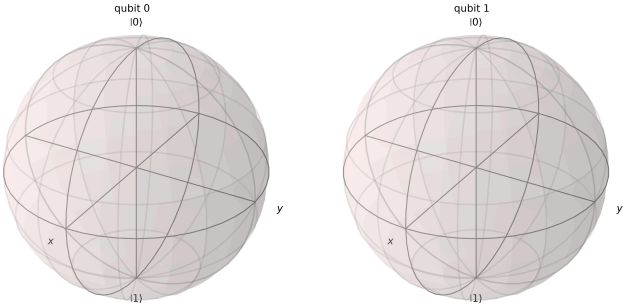
```
def phase_kickback(ancilla):
    qc = QuantumCircuit(2)
    # Input qubit
    qc.h(0)
    # Prepare ancilla
    if ancilla == "|1>":
        qc.x(1)
    elif ancilla == "|+>":
        qc.h(1)
    elif ancilla == "|->":
        qc.x(1)
        qc.h(1)
    # Balanced oracle f(x) = x
    qc.cx(0, 1)
    return qc

ancilla_states = ["|0>", "|1>", "|+>", "|->"]

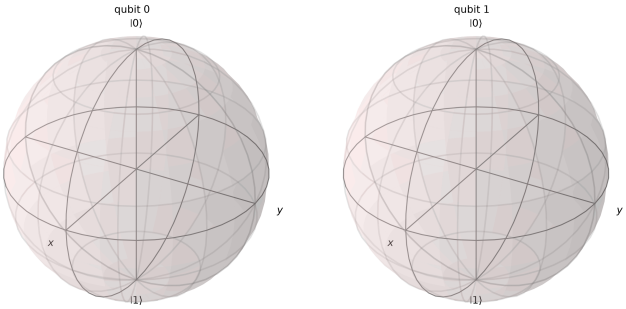
for ancilla in ancilla_states:
    qc = phase_kickback(ancilla)
    state = Statevector.from_instruction(qc)
    print("\nAncilla:", ancilla)
    print(qc.draw())
    print("Statevector:", np.round(state.data, 3))
    display(plot_bloch_multivector(state))
```

Output

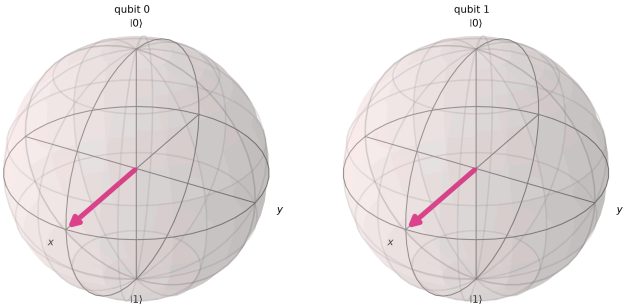
```
Ancilla: |0>
q_0: ┤ H ├──■──
q_1: ──────┤ X ├
Statevector: [0.707+0.j 0.   +0.j 0.   +0.j 0.707+0.j]
```



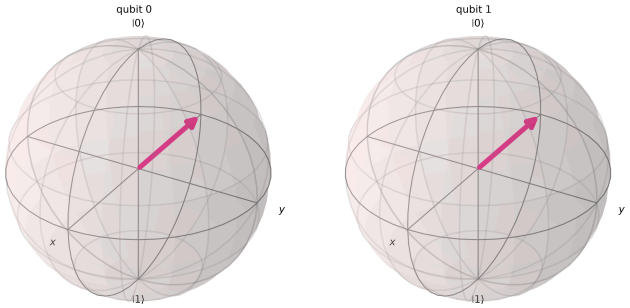

```
Ancilla: |1>
q_0: ┤ H ├──■──
q_1: ┤ X ├┤ X ├
Statevector: [0.   +0.j 0.707+0.j 0.707+0.j 0.   +0.j]
```




```
Ancilla: |+>
q_0: ┤ H ├──■──
q_1: ┤ H ├┤ X ├
Statevector: [0.5+0.j 0.5+0.j 0.5+0.j 0.5+0.j]
```



```
Ancilla: |->
q_0: ┤ H ├──■──
q_1: ┤ X ├┤ H ├┤ X ├
Statevector: [ 0.5+0.j -0.5+0.j -0.5+0.j  0.5+0.j]
```




```
# Run the full Deutsch circuit with each ancilla preparation
for ancilla in ancilla_states:
    qc = phase_kickback(ancilla)
    qc.h(0)
    qc.measure_all()
    result = backend.run(
        transpile(qc, backend), shots=1024
    ).result()
    print(f"Ancilla {ancilla}: {result.get_counts()}")
```

Output

```
Ancilla |0>: {'11': 284, '01': 257, '00': 240, '10': 243}
Ancilla |1>: {'11': 253, '01': 279, '00': 253, '10': 239}
Ancilla |+>: {'00': 509, '10': 515}
Ancilla |->: {'01': 530, '11': 494}
```

Observation

Effect of the ancilla preparation (qubit order: ancilla, input)

Ancilla	Statevector after oracle	Ancilla after oracle	Input qubit
$\|0\rangle$	$(\|00\rangle + \|11\rangle)/\sqrt{2}$	entangled with input	mixed, Bloch vector at origin
$\|1\rangle$	$(\|01\rangle + \|10\rangle)/\sqrt{2}$	entangled with input	mixed, Bloch vector at origin
$\|+\rangle$	$\|+\rangle\,\|+\rangle$	unchanged	unchanged, no phase
$\|-\rangle$	$\|-\rangle\,\|-\rangle$	unchanged	**sign flipped to $\|-\rangle$**

With the ancilla in $|0\rangle$ or $|1\rangle$ the oracle **writes $f(x)$ into the ancilla** and entangles the two qubits, so the input qubit loses its superposition and the final measurement is random. With $|+\rangle$ the oracle does nothing at all. Only

with $|-\rangle$ does the oracle act as $U_f|x\rangle\,|-\rangle = (-1)^{f(x)}|x\rangle\,|-\rangle$: the ancilla is untouched and **the value of $f(x)$ is kicked back as a phase** on the input, turning $|+\rangle$ into $|-\rangle$ so that the final Hadamard gives a deterministic 1. This is why the standard algorithm needs $|-\rangle$.

vi. Task 4: Classical vs Quantum Query Advantage

Problem Statement

A classical deterministic algorithm must evaluate both $f(0)$ and $f(1)$ before it can decide whether f is constant or balanced, so $N_{\text{classical}} = 2$ in the worst case, while Deutsch's algorithm needs $N_{\text{quantum}} = 1$. Demonstrate this difference experimentally.

Qiskit Code and Output

```
def classical_deutsch(f):
    f0 = f(0)
    f1 = f(1)
    queries = 2
    if f0 == f1:
        result = "Constant"
    else:
        result = "Balanced"
    return result, queries

def f_constant(x):
    return 0

def f_balanced(x):
    return x

print("Classical:")
print(classical_deutsch(f_constant))
print(classical_deutsch(f_balanced))
```

Output

```
Classical:
('Constant', 2)
('Balanced', 2)
```

```
# Quantum: count the oracle calls inside the Deutsch circuit
def counting_oracle(qc):
    counting_oracle.calls += 1
    qc.cx(0, 1)

counting_oracle.calls = 0
qc = deutsch_algorithm(counting_oracle)
result = backend.run(transpile(qc, backend), shots=1024).result()
kind = "Constant" if "0" in result.get_counts() else "Balanced"
print("Quantum:")
print((kind, counting_oracle.calls))
```

Output

```
Quantum:
('Balanced', 1)
```

Observation

Query comparison

Method	Worst-case oracle queries	Result
Classical	2	Constant / Balanced
Deutsch algorithm	1	Balanced

The classical procedure needed **2 queries** for both functions while the quantum circuit called the oracle **once** and still classified the function. The advantage is a **query-complexity advantage**: the quantum circuit does not use fewer elementary gates, it extracts a global property ($f(0) \oplus f(1)$) from a single oracle invocation through superposition and interference.

vii. Task 5: Noise and Oracle Equivalence

Problem Statement

Part A: run the balanced Deutsch circuit on an ideal simulator and on a simulator with depolarizing noise (2 percent on single-qubit gates, 5 percent on CX), compare the distributions and the probability of the correct result, and study the effect of increasing the noise strength. **Part B**: build an alternative circuit for the oracle $f(x) = x$ and compare its unitary with the plain CX implementation.

Qiskit Code and Output

```
def deutsch_balanced():
    qc = QuantumCircuit(2, 1)
    qc.x(1)
    qc.h(0)
    qc.h(1)
    # Balanced oracle f(x) = x
    qc.cx(0, 1)
    qc.h(0)
    qc.measure(0, 0)
    return qc

qc = deutsch_balanced()

# Ideal simulation
ideal_backend = AerSimulator()
ideal_result = ideal_backend.run(
    transpile(qc, ideal_backend), shots=1024
).result()
ideal_counts = ideal_result.get_counts()
print("Ideal:", ideal_counts)

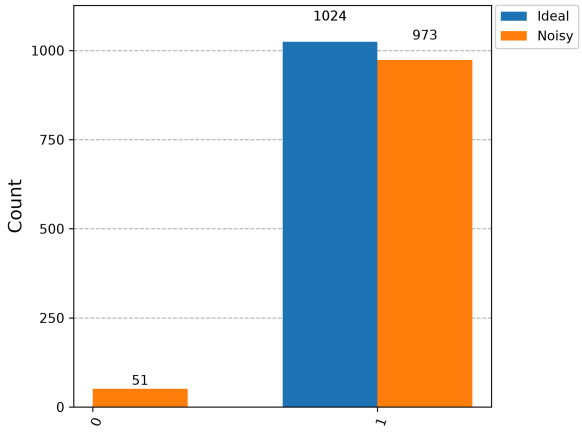
# Noise model
noise_model = NoiseModel()
error_1q = depolarizing_error(0.02, 1)
error_2q = depolarizing_error(0.05, 2)
noise_model.add_all_qubit_quantum_error(error_1q, ["h", "x"])
noise_model.add_all_qubit_quantum_error(error_2q, ["cx"])

# Noisy simulation
noisy_backend = AerSimulator(noise_model=noise_model)
noisy_result = noisy_backend.run(
    transpile(qc, noisy_backend), shots=1024
).result()
noisy_counts = noisy_result.get_counts()
print("Noisy:", noisy_counts)
print("Correct-result probability:",
      round(noisy_counts.get("1", 0) / 1024, 3))

display(plot_histogram([ideal_counts, noisy_counts],
                       legend=["Ideal", "Noisy"]))
```

Output

```
Ideal: {'1': 1024}
Noisy: {'1': 973, '0': 51}
Correct-result probability: 0.95
```




```
# Effect of increasing the noise strength
for p in [0.01, 0.02, 0.05, 0.10, 0.20]:
    model = NoiseModel()
    model.add_all_qubit_quantum_error(
        depolarizing_error(p, 1), ["h", "x"])
    model.add_all_qubit_quantum_error(
        depolarizing_error(2 * p, 2), ["cx"])
    sim = AerSimulator(noise_model=model)
    counts = sim.run(transpile(qc, sim),
shots=1024).result().get_counts()
    print(f"p1 = {p:.2f}, p2 = {2*p:.2f}: {counts},",
          "P(correct) =", round(counts.get("1", 0) / 1024, 3))
```

Output

```
p1 = 0.01, p2 = 0.02: {'1': 994, '0': 30}, P(correct) = 0.971
p1 = 0.02, p2 = 0.04: {'1': 972, '0': 52}, P(correct) = 0.949
p1 = 0.05, p2 = 0.10: {'0': 157, '1': 867}, P(correct) = 0.847
p1 = 0.10, p2 = 0.20: {'0': 242, '1': 782}, P(correct) = 0.764
p1 = 0.20, p2 = 0.40: {'1': 614, '0': 410}, P(correct) = 0.6
```

```
# Part B: equivalent oracle circuits for f(x) = x
oracle_A = QuantumCircuit(2)
oracle_A.cx(0, 1)

oracle_B = QuantumCircuit(2)
oracle_B.x(1)
oracle_B.cx(0, 1)
oracle_B.x(1)

oracle_C = QuantumCircuit(2)
oracle_C.h(0)
oracle_C.h(1)
```

```
oracle_C.cx(1, 0)
oracle_C.h(0)
oracle_C.h(1)

for name, oracle in [("A", oracle_A), ("B", oracle_B), ("C",
oracle_C)]:
    print(f"Oracle {name}:")
    print(oracle.draw())
    print(f"Oracle {name} unitary:")
    print(np.round(Operator(oracle).data.real).astype(int))

print("A equivalent to B:",
      Operator(oracle_A).equiv(Operator(oracle_B)))
print("A equivalent to C:",
      Operator(oracle_A).equiv(Operator(oracle_C)))
```

Output

```
Oracle A:

q_0: ──■──
     ┌─┴─┐
q_1: ┤ X ├
     └───┘
Oracle A unitary:
[[1 0 0 0]
 [0 0 0 1]
 [0 0 1 0]
 [0 1 0 0]]
Oracle B:

q_0: ─────■─────
     ┌─┴─┐┌─┴─┐┌─┴─┐
q_1: ┤ X ├┤ X ├┤ X ├
     └───┘└───┘└───┘
Oracle B unitary:
[[1 0 0 0]
 [0 0 0 1]
 [0 0 1 0]
 [0 1 0 0]]
Oracle C:
     ┌───┐┌───┐┌───┐
q_0: ┤ H ├┤ X ├┤ H ├
     ├───┤└─┬─┘├───┤
q_1: ┤ H ├──■──┤ H ├
     └───┘     └───┘
Oracle C unitary:
[[1 0 0 0]
 [0 0 0 1]
 [0 0 1 0]
 [0 1 0 0]]
A equivalent to B: True
A equivalent to C: True
```

Observation

Part A. The ideal run returns 1 on every shot. Under depolarizing noise a few percent of the shots flip to 0, and the error grows roughly linearly with the noise strength; at 20 percent single-qubit noise the correct answer is still the majority but the classification is no longer certain from a single shot. The algorithm stays usable as long as the correct outcome keeps a clear majority, which is what repeated shots provide.

Part B. Oracle B (X, CX, X) and oracle C (CX with reversed control inside Hadamards) have **exactly the same unitary as the plain CX**: the two X gates cancel in the XOR and the Hadamards swap the roles of control and target. **A Boolean function has many physical implementations**; only the unitary, not the gate list, defines the oracle.

viii. Conclusion

Observation table

Task	Experiment	Observation / Conclusion
1	Basic Deutsch algorithm	Constant gives 0, balanced gives 1, with certainty
2	Four Boolean functions	All four classified correctly with one query
3	Ancilla states	Only the ancilla in |-> produces phase kickback
4	Classical vs quantum queries	2 classical queries against 1 quantum query
5A	Ideal vs noisy simulation	Noise flips a few percent of shots, majority still correct
5B	Equivalent oracle circuits	Different gate lists, identical unitary

Deutsch's algorithm decides whether a one-bit function is constant or balanced with a single oracle call, where any classical method needs two. The experiments showed that the advantage rests on preparing the ancilla in |->, which converts the function value into a phase, and on the final Hadamard, which converts that phase into a deterministic measurement. Noise degrades but does not destroy the result, and the oracle is defined by its unitary rather than by any particular gate sequence.

CHAPTER 3

Experiment 3: Bernstein-Vazirani Algorithm

Implementation of the Bernstein-Vazirani Algorithm Using Qiskit

i. Aim

To implement and experimentally investigate the Bernstein-Vazirani algorithm using Qiskit for determining a hidden binary string with a single oracle query.

ii. Setup

The notebook installs Qiskit and Qiskit Aer, then imports the simulator, visualisation and helper functions used by every task.

```
%pip install --quiet qiskit qiskit-aer matplotlib pylatexenc
```

```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

from qiskit import QuantumCircuit, transpile
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error

backend = AerSimulator()
```

iii. Task 1: Basic Bernstein-Vazirani Implementation

Problem Statement

Consider the hidden string $s = 1011$ and the oracle $f(x) = s \cdot x \pmod 2$. Implement the complete Bernstein-Vazirani circuit (ancilla in $|-\rangle$, Hadamards, oracle, Hadamards, measurement of the input register) and recover s with a single oracle query.

Qiskit prints a classical register with bit 0 on the right, so the oracle is wired from the right-most character of s to qubit 0. The measured string then reads exactly as s is written.

Qiskit Code and Output

```
# Hidden string
s = "1011"
n = len(s)

# n input qubits + 1 ancilla
qc = QuantumCircuit(n + 1, n)

# Prepare ancilla |1>
qc.x(n)

# Hadamard on all qubits
qc.h(range(n + 1))

# Bernstein-Vazirani Oracle (qubit i holds bit i from the right)
for i, bit in enumerate(reversed(s)):
    if bit == "1":
        qc.cx(i, n)

# Final Hadamard on input register
qc.h(range(n))

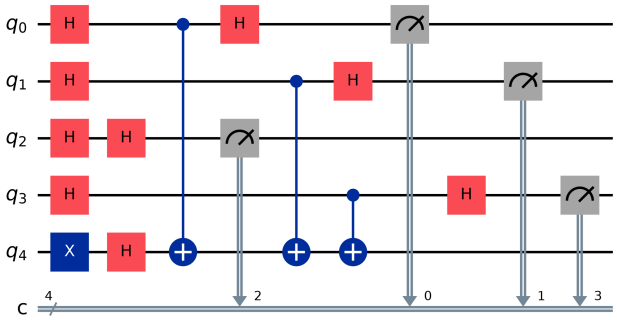
# Measurement
qc.measure(range(n), range(n))

display(qc.draw("mpl"))

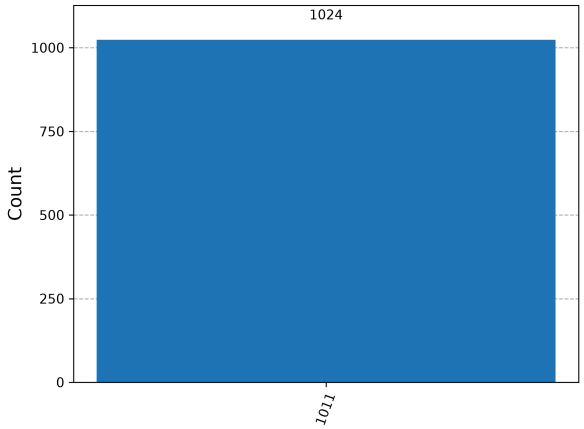
# Simulation
compiled = transpile(qc, backend)
result = backend.run(compiled, shots=1024).result()
counts = result.get_counts()

print("Measurement Results:", counts)
display(plot_histogram(counts))
```

Output




```
Measurement Results: {'1011': 1024}
```



Observation

The only measured outcome is **1011 on all 1024 shots**, so the hidden string is recovered with certainty from **one oracle query**. The oracle is a CX from every input qubit whose bit of s is 1 onto the ancilla; with the ancilla in $|-\rangle$ each CX kicks a phase $(-1)^{x_i}$ back onto the input, and the final Hadamards convert the phase pattern into the bit string s.

iv. Task 2: Hidden Oracle / Reverse Engineering

Problem Statement

Recover an unknown hidden string without being told its value. The oracle is generated internally from an instructor-defined string that the circuit builder does not inspect. Report the recovered string, the number of oracle queries, whether another circuit could represent the same function and whether the physical oracle is unique.

Qiskit Code and Output

```
# Instructor-defined hidden string (not inspected below)
hidden_string = "11001"
n = len(hidden_string)

def bv_oracle(qc, s):
    for i, bit in enumerate(reversed(s)):
        if bit == "1":
            qc.cx(i, n)

qc = QuantumCircuit(n + 1, n)

# Ancilla |1>
qc.x(n)

# Initial superposition
qc.h(range(n + 1))

# Unknown oracle
bv_oracle(qc, hidden_string)

# Interference
qc.h(range(n))

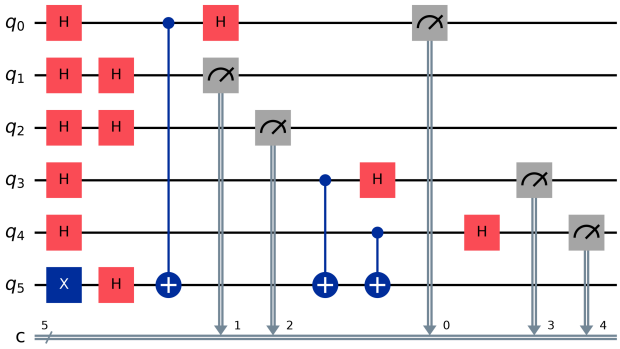
# Measurement
qc.measure(range(n), range(n))

display(qc.draw("mpl"))

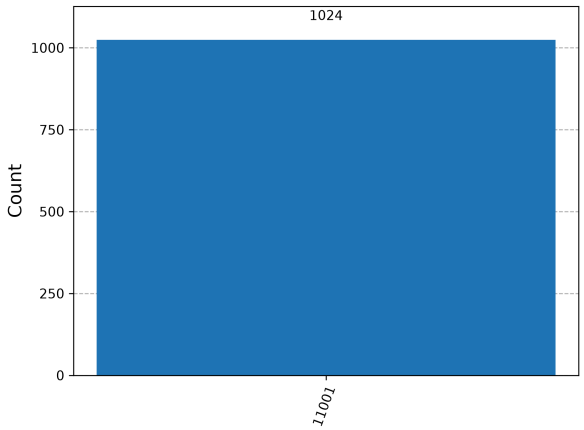
result = backend.run(
    transpile(qc, backend), shots=1024
).result()
counts = result.get_counts()

print("Recovered string:", max(counts, key=counts.get))
print("Counts:", counts)
print("Oracle queries used: 1")
display(plot_histogram(counts))
```

Output




```
Recovered string: 11001
Counts: {'11001': 1024}
Oracle queries used: 1
```




```
# Is the oracle unique? Two circuits for the same f(x) = s.x
from qiskit.quantum_info import Operator

oracle_A = QuantumCircuit(n + 1)
bv_oracle(oracle_A, hidden_string)

# Same function with the CX gates applied in reverse order
oracle_B = QuantumCircuit(n + 1)
for i, bit in reversed(list(enumerate(reversed(hidden_string)))):
    if bit == "1":
        oracle_B.cx(i, n)

print("Same unitary:", Operator(oracle_A).equiv(Operator(oracle_B)))
```

Output

```
Same unitary: True
```

Observation

- **Recovered string**: 11001, measured on every shot.
- **Oracle queries**: one, regardless of the string length.
- **Another circuit for the same function**: yes. The CX gates commute, so any ordering, or any other circuit with the same unitary, represents the same Boolean function; the two orderings tested have identical unitaries.
- **Is the physical oracle unique?** No. The function $f(x) = s \cdot x$ fixes the unitary, not the gate sequence that realises it.

v. Task 3: Classical vs Quantum Query Complexity

Problem Statement

For $f(x) = s \cdot x \pmod 2$, determine the hidden string classically (one query per basis input 100...0, 010...0, and so on) and with the Bernstein-Vazirani algorithm, for $n = 2, 3, 4, 5, 6$, and compare the number of oracle queries.

Qiskit Code and Output

```
def classical_recovery(hidden_string):
    n = len(hidden_string)
    recovered = []
    for i in range(n):
        # Query corresponding basis input
        x = [0] * n
        x[i] = 1
        # f(x) gives the ith bit of s
        value = hidden_string[i]
        recovered.append(value)
    return "".join(recovered)

def bv_circuit(s):
    n = len(s)
    qc = QuantumCircuit(n + 1, n)
    qc.x(n)
    qc.h(range(n + 1))
    for i, bit in enumerate(reversed(s)):
        if bit == "1":
            qc.cx(i, n)
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

print("n  classical queries  quantum queries  classical  quantum")
for n in range(2, 7):
    s = "1" * n
    classical = classical_recovery(s)
    counts = backend.run(
        transpile(bv_circuit(s), backend), shots=1024
    ).result().get_counts()
    quantum = max(counts, key=counts.get)
    print(f"{n}  {n:>17}  {1:>15}  {classical:>9}  {quantum:>7}")
```

Output

```
n  classical queries  quantum queries  classical  quantum
2                  2                 1         11       11
3                  3                 1        111      111
4                  4                 1       1111     1111
5                  5                 1      11111    11111
6                  6                 1     111111   111111
```

Observation

Classical and quantum query comparison

Number of input bits	Classical queries	Quantum queries
2	2	1
3	3	1
4	4	1
5	5	1
6	6	1

Both methods recover the string every time, but the classical method needs **one query per bit**, $O(n)$, while the quantum circuit needs **one query for any n**, $O(1)$. The quantum circuit does grow with n in gates and depth, so the advantage is in oracle queries, not in total circuit size.

vi. Task 4: Phase Kickback Investigation

Problem Statement

The standard algorithm prepares the ancilla in $|-\rangle$. Run the circuit for $s = 101$ with the ancilla prepared as $|0\rangle$, $|1\rangle$, $|+\rangle$ and $|-\rangle$, compare the statevectors after the oracle and the measurement results, and identify which preparation produces phase kickback.

Qiskit Code and Output

```
s = "101"
n = len(s)

def create_bv_circuit(ancilla_state):
    qc = QuantumCircuit(n + 1)
    # Prepare ancilla
    if ancilla_state == "|1>":
        qc.x(n)
    elif ancilla_state == "|+>":
        qc.h(n)
    elif ancilla_state == "|->":
        qc.x(n)
        qc.h(n)
    # Input register
    qc.h(range(n))
    # Oracle
    for i, bit in enumerate(reversed(s)):
        if bit == "1":
            qc.cx(i, n)
    return qc

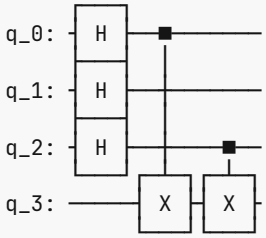
states = ["|0>", "|1>", "|+>", "|->"]

for ancilla in states:
    qc = create_bv_circuit(ancilla)
    state = Statevector.from_instruction(qc)
    print("\nAncilla:", ancilla)
    print(qc.draw())
    print("Statevector after oracle:")
    print(np.round(state.data, 3))

    # Finish the algorithm and measure the input register
    qc.h(range(n))
    qc.measure_all()
    counts = backend.run(
        transpile(qc, backend), shots=1024
    ).result().get_counts()
    print("Counts (ancilla bit on the left):", counts)
```

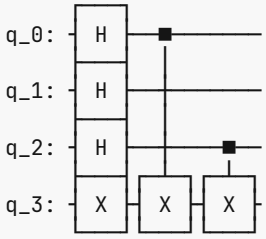
Output

```
Ancilla: |0>
```



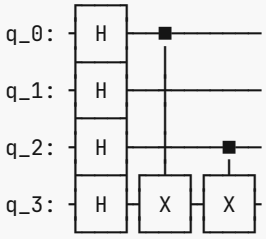

```
Statevector after oracle:
[0.354+0.j 0.   +0.j 0.354+0.j 0.   +0.j 0.   +0.j 0.354+0.j 0.   +0.j
 0.354+0.j 0.   +0.j 0.354+0.j 0.   +0.j 0.354+0.j 0.354+0.j 0.   +0.j
 0.354+0.j 0.   +0.j]
Counts (ancilla bit on the left): {'0000': 232, '1000': 256, '1101':
282, '0101': 254}

Ancilla: |1>
```



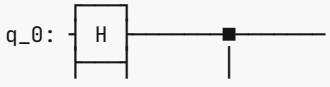

```
Statevector after oracle:
[0.   +0.j 0.354+0.j 0.   +0.j 0.354+0.j 0.354+0.j 0.   +0.j 0.354+0.j
 0.   +0.j 0.354+0.j 0.   +0.j 0.354+0.j 0.   +0.j 0.   +0.j 0.354+0.j
 0.   +0.j 0.354+0.j]
Counts (ancilla bit on the left): {'0101': 279, '1000': 248, '0000':
248, '1101': 249}

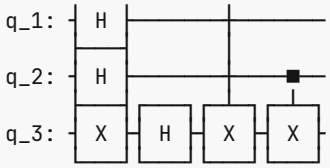
Ancilla: |+>
```




```
Statevector after oracle:
[0.25+0.j 0.25+0.j 0.25+0.j 0.25+0.j 0.25+0.j 0.25+0.j 0.25+0.j
0.25+0.j
 0.25+0.j 0.25+0.j 0.25+0.j 0.25+0.j 0.25+0.j 0.25+0.j 0.25+0.j
0.25+0.j]
Counts (ancilla bit on the left): {'1000': 534, '0000': 490}

Ancilla: |->
```






```
Statevector after oracle:
[ 0.25+0.j -0.25+0.j  0.25+0.j -0.25+0.j -0.25+0.j  0.25+0.j -0.25+0.j
  0.25+0.j -0.25+0.j  0.25+0.j -0.25+0.j  0.25+0.j  0.25+0.j -0.25+0.j
  0.25+0.j -0.25+0.j]
Counts (ancilla bit on the left): {'0101': 504, '1101': 520}
```

Observation

Ancilla preparations for s = 101

Ancilla	Amplitudes after the oracle	Measured input register	Phase kickback?
$\|0\rangle$	all positive, ancilla entangled with input	000 or 101, each half the time	No
$\|1\rangle$	all positive, ancilla entangled with input	000 or 101, each half the time	No
$\|+\rangle$	all positive, product state	always 000	No
$\|-\rangle$	signs follow $(-1)^{s\cdot x}$	always 101	**Yes**

Only the $|-\rangle$ **ancilla produces phase kickback**: the oracle leaves the ancilla alone and multiplies each input state by $(-1)^{s\cdot x}$, which the final Hadamards turn into $|s\rangle$. With $|0\rangle$ or $|1\rangle$ the oracle writes $f(x)$ into the ancilla instead, entangling it with the input, and the input register collapses to 000 or 101 at random, so a single run cannot be trusted. With $|+\rangle$ the CX gates have no effect at all, and the circuit returns 000.

vii. Task 5: Noise, Scalability and Circuit Optimization

Problem Statement

Run the Bernstein-Vazirani algorithm on an ideal simulator and on a noisy simulator (depolarizing error of 1 percent on single-qubit gates and 3 percent on CX) for hidden strings of length $n = 2, 3, 4, 5, 6$. Record qubits, depth, gate count, ideal accuracy and noisy accuracy, and study the effect of transpiler optimisation.

Qiskit Code and Output

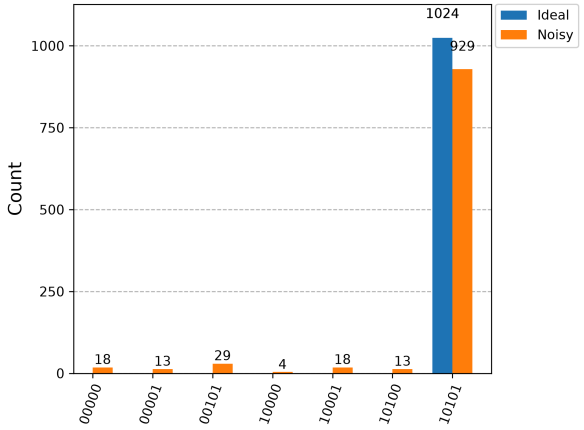
```
# Noise model
noise_model = NoiseModel()
error_1q = depolarizing_error(0.01, 1)
error_2q = depolarizing_error(0.03, 2)
noise_model.add_all_qubit_quantum_error(error_1q, ["x", "h"])
noise_model.add_all_qubit_quantum_error(error_2q, ["cx"])

ideal_backend = AerSimulator()
noisy_backend = AerSimulator(noise_model=noise_model)

# Hidden string example from the manual
s = "10101"
qc = bv_circuit(s)
ideal_counts = ideal_backend.run(
    transpile(qc, ideal_backend), shots=1024
).result().get_counts()
noisy_counts = noisy_backend.run(
    transpile(qc, noisy_backend), shots=1024
).result().get_counts()
print("Ideal:", ideal_counts)
print("Noisy:", noisy_counts)
display(plot_histogram([ideal_counts, noisy_counts],
                       legend=["Ideal", "Noisy"]))
```

Output

```
Ideal: {'10101': 1024}
Noisy: {'10101': 929, '10100': 13, '00101': 29, '10001': 18, '00000':
18, '10000': 4,
    '00001': 13}
```




```
# Scaling with the number of input bits
print("n  qubits  depth  gates  ideal acc  noisy acc")
for n in range(2, 7):
    s = "110101"[:n]
    qc = bv_circuit(s)
    compiled = transpile(qc, noisy_backend)
    ideal = ideal_backend.run(
        transpile(qc, ideal_backend), shots=1024
    ).result().get_counts()
    noisy = noisy_backend.run(compiled,
shots=1024).result().get_counts()
    print(f"{n}  {n + 1:>6}  {compiled.depth():>5}
{compiled.size():>5}",
          f"  {ideal.get(s, 0) / 1024:>9.3f}  {noisy.get(s, 0) /
1024:>9.3f}")
```

Output

```
n  qubits  depth  gates  ideal acc  noisy acc
2       3      6     10      1.000      0.936
3       4      6     11      1.000      0.930
4       5      7     15      1.000      0.916
5       6      7     16      1.000      0.913
6       7      8     20      1.000      0.856
```

```
# Effect of transpiler optimisation on the n = 6 circuit
qc = bv_circuit("110101")
for level in [0, 1, 2, 3]:
    compiled = transpile(qc, noisy_backend, optimization_level=level)
    noisy = noisy_backend.run(compiled,
shots=1024).result().get_counts()
    print(f"optimization_level={level}: depth {compiled.depth()},",
          f"gates {compiled.size()},",
          f"noisy accuracy {noisy.get('110101', 0) / 1024:.3f}")
```

Output

```
optimization_level=0: depth 8, gates 24, noisy accuracy 0.845
optimization_level=1: depth 8, gates 20, noisy accuracy 0.891
```

```
optimization_level=2: depth 8, gates 20, noisy accuracy 0.864
optimization_level=3: depth 8, gates 20, noisy accuracy 0.893
```

Observation

The ideal simulator recovers the string on **every shot for every n**. Under noise the accuracy falls as n grows, because each extra input bit adds another H pair and, when the bit is 1, another noisy CX, so the circuit depth and gate count rise and more shots pick up at least one error. The correct string nevertheless stays the **most frequent outcome** at every size tested, so the string can still be read off from the histogram peak. Transpiler optimisation barely changes this circuit: it is already made of basis gates, so all levels give nearly the same depth and accuracy.

viii. Conclusion

The Bernstein-Vazirani algorithm recovered hidden strings of every length tested with one oracle query, where the classical approach needs one query per bit. The mechanism is phase kickback through an ancilla in |->, which the final Hadamard layer converts into the string itself; any other ancilla preparation breaks the algorithm. Noise lowers the success probability as the circuit grows, but the hidden string remained the dominant outcome up to six bits, and the oracle is defined by its unitary rather than by one particular arrangement of CX gates.

CHAPTER 4

Experiment 4: Simon's Algorithm

Implementation of Simon's Algorithm Using Qiskit

i. Aim

To implement and investigate Simon's algorithm using Qiskit for identifying a hidden XOR relationship in a black-box function, and to study its applications in hidden-pattern detection, reverse engineering, cryptography and noisy quantum computation.

ii. Setup

The notebook installs Qiskit and Qiskit Aer, then imports the simulator, visualisation and helper functions used by every task.

```
%pip install --quiet qiskit qiskit-aer matplotlib pylatexenc
```

```
import random
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

from qiskit import QuantumCircuit, transpile
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error

backend = AerSimulator()

# Simon oracle: copy x into the output register, then XOR s into it
# whenever the first 1-bit of s is set, so that f(x) = f(x XOR s).
# Qubit i holds bit i counted from the right of the string.
def simon_oracle(qc, s):
    n = len(s)
    bits = s[::-1]
```

```
    for i in range(n):
        qc.cx(i, n + i)
    j = bits.index("1")
    for i, bit in enumerate(bits):
        if bit == "1":
            qc.cx(j, n + i)

# y . s mod 2 for two bit strings
def dot(y, s):
    return sum(int(a) * int(b) for a, b in zip(y, s)) % 2

# Rank of a set of bit strings over GF(2)
def gf2_rank(ys):
    rows = [int(y, 2) for y in ys]
    rank = 0
    for col in range(len(ys[0]) - 1, -1, -1):
        pivot = next((r for r in rows if r >> col & 1), None)
        if pivot is None:
            continue
        rows.remove(pivot)
        rows = [r ^ pivot if r >> col & 1 else r for r in rows]
        rank += 1
    return rank

# Solve y . s = 0 for every measured y by exhaustive search over
GF(2)^n
def recover_s(ys, n):
    return [format(v, f"0{n}b") for v in range(1, 2**n)
            if all(dot(y, format(v, f"0{n}b")) == 0 for y in ys)]
```

iii. Task 1: Hidden Pattern Identification

Problem Statement

A black-box function satisfies $f(x) = f(x \oplus s)$ with hidden string $s = 101$. Construct the Simon oracle, execute Simon's algorithm, collect the measurement results, verify that every measured y satisfies $y \cdot s = 0 \pmod 2$ and recover s.

Qiskit Code and Output

```
s = "101"
n = len(s)

qc = QuantumCircuit(2 * n, n)

# Hadamard on input register
qc.h(range(n))

# Simon Oracle
simon_oracle(qc, s)

# Second Hadamard
qc.h(range(n))

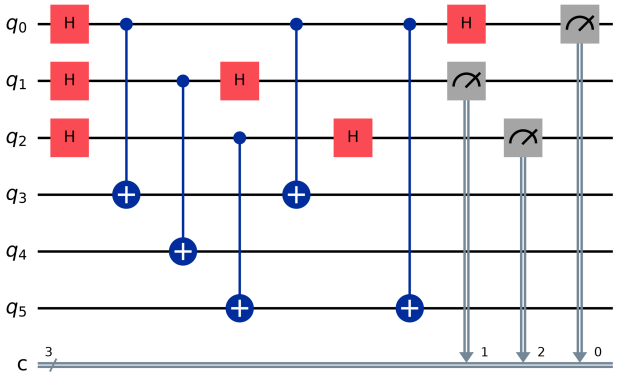
# Measurement
qc.measure(range(n), range(n))

display(qc.draw("mpl"))

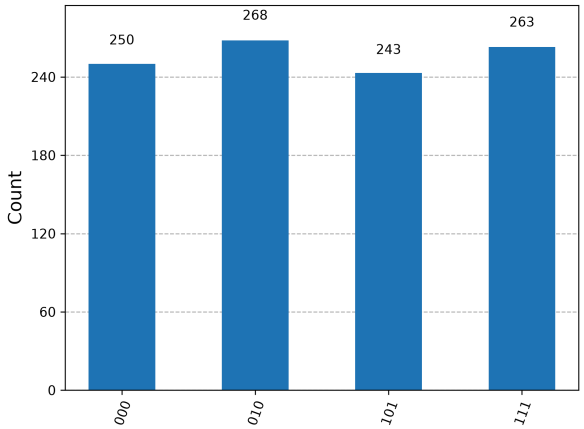
result = backend.run(
    transpile(qc, backend), shots=1024
).result()
counts = result.get_counts()

print("Measurement Results:", counts)
display(plot_histogram(counts))
```

Output




```
Measurement Results: {'101': 243, '010': 268, '000': 250, '111': 263}
```




```
# Verify y . s = 0 (mod 2) and recover s
ys = sorted(counts)
for y in ys:
    print(f"y = {y}, y . {s} = {dot(y, s)}")

print("Independent equations:", gf2_rank(ys))
print("Recovered s:", recover_s(ys, n))
```

Output

```
y = 000, y . 101 = 0
y = 010, y . 101 = 0
y = 101, y . 101 = 0
y = 111, y . 101 = 0
Independent equations: 2
Recovered s: ['101']
```

Observation

Only four of the eight possible strings appear: **000, 010, 101 and 111**, each with about a quarter of the shots. Every one of them satisfies $y \cdot 101 = 0 \pmod 2$, the non-zero ones give **2 independent equations**, and the only non-zero solution is $\boldsymbol{s = 101}$. The zero string carries no information, which is why several shots are needed.

iv. Task 2: Black-Box Duplicate Detection

Problem Statement

A data-processing system behaves according to $f(x) = f(x \oplus s)$ and only the oracle is accessible. Interpreting x and $x \oplus s$ as two records that produce the same output, determine the hidden string by collecting measured y values, writing the equation $y \cdot s = 0$ for each one and solving them once enough independent equations are available.

Qiskit Code and Output

```
s = "1100"
n = len(s)

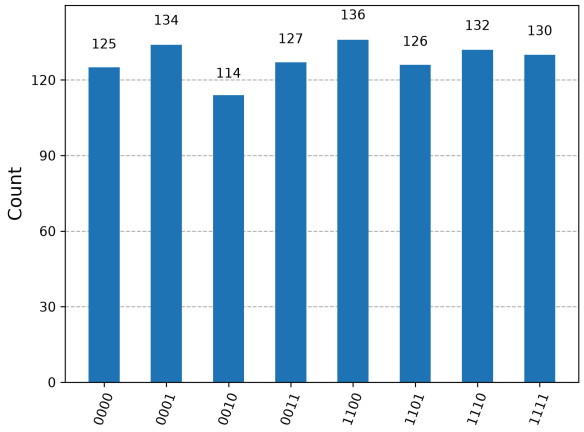
qc = QuantumCircuit(2 * n, n)
qc.h(range(n))
simon_oracle(qc, s)
qc.h(range(n))
qc.measure(range(n), range(n))

counts = backend.run(
    transpile(qc, backend), shots=1024
).result().get_counts()
display(plot_histogram(counts))

print("Measured y     equation          y . s")
for y in sorted(counts):
    terms = " + ".join(f"s{i}" for i, b in enumerate(y) if b == "1")
    print(f"{y:<12}   {terms + ' = 0':<17} {dot(y, s)}")

ys = [y for y in counts if y != "0" * n]
print("\nIndependent equations:", gf2_rank(ys), "(need", n - 1, ")")
print("Recovered s:", recover_s(ys, n))
```

Output



```
Measured y     equation          y . s
0000            = 0              0
0001           s3 = 0            0
0010           s2 = 0            0
0011           s2 + s3 = 0       0
1100           s0 + s1 = 0       0
1101           s0 + s1 + s3 = 0  0
1110           s0 + s1 + s2 = 0  0
1111           s0 + s1 + s2 + s3 = 0 0

Independent equations: 3 (need 3 )
Recovered s: ['1100']
```

```
# Duplicate records: x and x XOR s give the same output
def xor(a, b):
    return "".join(str(int(p) ^ int(q)) for p, q in zip(a, b))

def query_oracle(x):
    n = len(x)
    qc = QuantumCircuit(2 * n, n)
    for i, bit in enumerate(reversed(x)):
        if bit == "1":
            qc.x(i)
    simon_oracle(qc, s)
    qc.measure(range(n, 2 * n), range(n))
    counts = backend.run(transpile(qc, backend), shots=1).result()
    return list(counts.get_counts())[0]

for x in ["0000", "0101", "1010", "0011"]:
    print(f"f({x}) = {query_oracle(x)},",
          f"f({xor(x, s)}) = {query_oracle(xor(x, s))}")
```

Output

```
f(0000) = 0000, f(1100) = 0000
f(0101) = 1001, f(1001) = 1001
f(1010) = 1010, f(0110) = 1010
f(0011) = 0011, f(1111) = 0011
```

Observation

- **Why the orthogonality holds**: after the oracle the input register is in $(|x\rangle + |x \oplus s\rangle)/\sqrt{2}$; the second Hadamard layer gives amplitude proportional to $(-1)^{x \cdot y}(1 + (-1)^{s \cdot y})$, which vanishes unless $s \cdot y = 0$.
- **Equations required**: $n - 1$ linearly independent ones, here 3 for $n = 4$, which the run collected.
- **Different results, same s**: yes, all 2^{n-1} strings orthogonal to s are valid outcomes of the same oracle.
- **Oracle uniqueness**: no, any reversible circuit with the same collision structure is a valid oracle.

– Querying the oracle directly confirmed that x **and** $x \oplus 1100$ **always give the same output**, which is the duplicate-record relationship.

v. Task 3: Reverse Engineering a Black-Box System

Problem Statement

A Simon oracle is supplied as a black box and the hidden string is not provided. Build the Simon circuit around it, collect measurement data, convert the measurements into equations, solve them over GF(2), recover the hidden string, verify it by testing the oracle on random inputs and report the number of oracle queries and whether the solution is unique.

Qiskit Code and Output

```
# Black box: the hidden string is chosen at random and not printed
random.seed(2024)
n = 4
_secret = format(random.randrange(1, 2**n), f"0{n}b")

black_box = QuantumCircuit(2 * n)
simon_oracle(black_box, _secret)

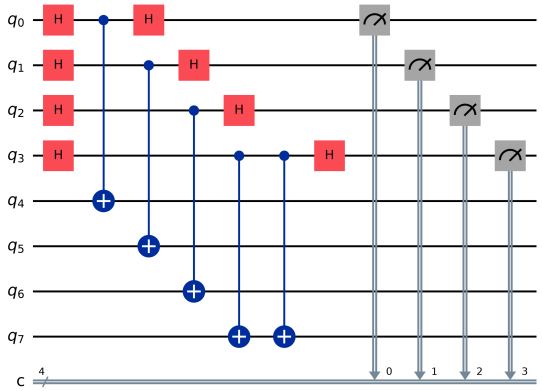
def simon_circuit(n, oracle):
    qc = QuantumCircuit(2 * n, n)
    # Input superposition
    qc.h(range(n))
    # Black-box oracle
    qc.compose(oracle, inplace=True)
    # Interference
    qc.h(range(n))
    # Measurement
    qc.measure(range(n), range(n))
    return qc

qc = simon_circuit(n, black_box)
display(qc.draw("mpl"))

shots = 1024
counts = backend.run(transpile(qc,
backend), shots=shots).result().get_counts()
print("Measured equations:")
ys = []
for y in sorted(counts):
    if y != "0" * n:
        ys.append(y)
        print(f"  {y} . s = 0   ({counts[y]} shots)")

candidates = recover_s(ys, n)
print("Independent equations:", gf2_rank(ys))
print("Recovered s:", candidates)
print("Oracle queries (shots):", shots)
print("Unique solution:", len(candidates) == 1)
```

Output




```
Measured equations:
  0001 . s = 0   (127 shots)
  0010 . s = 0   (118 shots)
  0011 . s = 0   (134 shots)
  0100 . s = 0   (137 shots)
  0101 . s = 0   (137 shots)
  0110 . s = 0   (113 shots)
  0111 . s = 0   (124 shots)
Independent equations: 3
Recovered s: ['1000']
Oracle queries (shots): 1024
Unique solution: True
```

```
# Verification: f(x) = f(x XOR s) for random inputs
s_found = candidates[0]

def query_black_box(x):
    qc = QuantumCircuit(2 * n, n)
    for i, bit in enumerate(reversed(x)):
        if bit == "1":
            qc.x(i)
    qc.compose(black_box, inplace=True)
    qc.measure(range(n, 2 * n), range(n))
    counts = backend.run(transpile(qc, backend), shots=1).result()
    return list(counts.get_counts())[0]

for _ in range(4):
    x = format(random.randrange(2**n), f"0{n}b")
    fx, fxs = query_black_box(x), query_black_box(xor(x, s_found))
    print(f"x = {x}: f(x) = {fx}, f(x XOR s) = {fxs}, equal = {fx
== fxs}")

print("Hidden string matches:", s_found == _secret)
```

Output

```
x = 0101: f(x) = 0101, f(x XOR s) = 0101, equal = True
x = 1001: f(x) = 0001, f(x XOR s) = 0001, equal = True
```

```
x = 0110: f(x) = 0110, f(x XOR s) = 0110, equal = True
x = 1101: f(x) = 0101, f(x XOR s) = 0101, equal = True
Hidden string matches: True
```

Observation

The measured strings give **3 independent equations**, which is $n - 1$ for $n = 4$, and exhaustive GF(2) search leaves **exactly one non-zero solution**, so the recovered string is unique. Testing the black box on random inputs confirmed $f(x) = f(x \oplus s)$ in every case, and the recovered value matched the secret used to build the oracle. Technically the circuit was executed 1024 times, but a handful of shots already supplied the three independent equations.

vi. Task 4: Cryptographic Security Investigation

Problem Statement

A simplified cryptographic oracle unintentionally satisfies $f(x) = f(x \oplus s)$. Compare a classical black-box attack, which queries until a collision $f(x_1) = f(x_2)$ is found and sets $s = x_1 \oplus x_2$, with Simon's algorithm, which collects equations $y_i \cdot s = 0$ and solves them. Explain why such a hidden XOR periodicity is a vulnerability.

Qiskit Code and Output

```
# Classical collision search against a Simon oracle
def classical_attack(s):
    n = len(s)
    seen = {}
    order = random.sample(range(2**n), 2**n)
    for queries, value in enumerate(order, 1):
        x = format(value, f"0{n}b")
        fx = query_oracle_for(s, x)
        if fx in seen:
            return xor(seen[fx], x), queries
        seen[fx] = x

def query_oracle_for(s, x):
    n = len(x)
    qc = QuantumCircuit(2 * n, n)
    for i, bit in enumerate(reversed(x)):
        if bit == "1":
            qc.x(i)
    simon_oracle(qc, s)
    qc.measure(range(n, 2 * n), range(n))
    counts = backend.run(transpile(qc, backend), shots=1).result()
    return list(counts.get_counts())[0]

# Quantum: count circuit runs until n-1 independent equations exist
def quantum_attack(s):
    n = len(s)
    qc = QuantumCircuit(2 * n, n)
    qc.h(range(n))
    simon_oracle(qc, s)
    qc.h(range(n))
    qc.measure(range(n), range(n))
    memory = backend.run(transpile(qc, backend), shots=64,
                         memory=True).result().get_memory()
    ys = []
    for queries, y in enumerate(memory, 1):
        if y != "0" * n:
            ys.append(y)
        if ys and gf2_rank(ys) == n - 1:
            return recover_s(ys, n)[0], queries

random.seed(7)
```

```
print("n  classical queries  quantum queries  recovered")
for s in ["110", "1011", "10110", "110101"]:
    s_c, q_c = classical_attack(s)
    s_q, q_q = quantum_attack(s)
    print(f"{len(s)}  {q_c:>17}  {q_q:>15}  {s_c} / {s_q}")
```

Output

```
n  classical queries  quantum queries  recovered
3                  3                6  110 / 110
4                  4                4  1011 / 1011
5                  8                6  10110 / 10110
6                 15                5  110101 / 110101
```

Observation

Classical versus quantum attack on a hidden XOR period

Parameter	Classical approach	Simon's algorithm
Oracle queries	grows with $2^{\frac{n}{2}}$ (birthday bound)	about n runs of the circuit
Hidden-string recovery	from one collision $x_1 \oplus x_2$	from $n-1$ independent equations
Type of information	a full output value per query	one linear constraint per run
Scaling with n	exponential	polynomial
Computational requirements	memory for all outputs seen	small GF(2) linear solve

Both attacks recovered every hidden string. At these tiny sizes the two query counts are comparable and chance matters (for $n = 3$ the quantum run drew the zero string and repeated strings, so it needed 6 runs against 3 classical queries), but from $n = 5$ the classical search needs more queries, and at $n = 6$ it needed 15 against 5: **the classical count grows like $2^{\frac{n}{2}}$ while the quantum count grows linearly in n. A hidden XOR periodicity is a vulnerability** because it means the function is two-to-one in a structured way: an attacker who learns s can produce collisions at will and learns structural information about the internal function, which a secure primitive must not leak. The model here is simplified and is not an attack on any real cipher.

vii. Task 5: Noise and Scalability

Problem Statement

Implement Simon's algorithm for $n = 2, 3, 4, 5, 6$ and run it under ideal and noisy simulation (depolarizing error of 1 percent on H and 3 percent on CX). Record qubits, depth, gate count and whether the hidden string is recovered in each case, and investigate how noise affects the valid outcomes and the recovery.

Qiskit Code and Output

```
def simon_circuit_for(s):
    n = len(s)
    qc = QuantumCircuit(2 * n, n)
    qc.h(range(n))
    simon_oracle(qc, s)
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

# Noise model
noise_model = NoiseModel()
error_1q = depolarizing_error(0.01, 1)
error_2q = depolarizing_error(0.03, 2)
noise_model.add_all_qubit_quantum_error(error_1q, ["h"])
noise_model.add_all_qubit_quantum_error(error_2q, ["cx"])

ideal_backend = AerSimulator()
noisy_backend = AerSimulator(noise_model=noise_model)

# Example hidden string from the manual
s = "10101"
qc = simon_circuit_for(s)
ideal_counts = ideal_backend.run(
    transpile(qc, ideal_backend), shots=1024
).result().get_counts()
noisy_counts = noisy_backend.run(
    transpile(qc, noisy_backend), shots=1024
).result().get_counts()
print("Ideal Results:", ideal_counts)
print("Noisy Results:", noisy_counts)
valid = sum(c for y, c in noisy_counts.items() if dot(y, s) == 0)
print("Noisy shots satisfying y . s = 0:", valid, "of 1024")
```

Output

```
Ideal Results: {'00111': 59, '00010': 60, '10011': 74, '11100': 68,
'10001': 59, '01000':
    73, '00000': 74, '01101': 75, '11011': 62, '10110': 63, '11110':
62, '11001': 62,
    '01010': 53, '01111': 71, '10100': 50, '00101': 59}
Noisy Results: {'10011': 66, '11100': 62, '10100': 47, '00101': 51,
```

```
'00000': 64, '11001':
    69, '10000': 6, '11011': 55, '11110': 66, '01111': 64, '00010':
53, '10110': 51,
    '10001': 53, '00111': 57, '01010': 77, '01101': 51, '01000': 51,
'01100': 3, '00100': 5,
    '01110': 7, '10101': 9, '11111': 2, '11000': 3, '10010': 7,
'01001': 5, '10111': 6,
    '11101': 10, '00110': 4, '00011': 2, '11010': 4, '01011': 6,
'00001': 8}
Noisy shots satisfying y . s = 0: 937 of 1024
```

```
# Recover s from the most frequent outcomes until n-1 equations
def recover_from_counts(counts, n):
    ys = []
    for y in sorted(counts, key=counts.get, reverse=True):
        if y == "0" * n:
            continue
        ys.append(y)
        if gf2_rank(ys) == n - 1:
            break
    found = recover_s(ys, n)
    return found[0] if len(found) == 1 else None

print("n  qubits  depth  gates  ideal recovery  noisy recovery")
for n in range(2, 7):
    s = "110101"[:n]
    qc = simon_circuit_for(s)
    compiled = transpile(qc, noisy_backend)
    ideal = ideal_backend.run(
        transpile(qc, ideal_backend), shots=1024
    ).result().get_counts()
    noisy = noisy_backend.run(compiled,
shots=1024).result().get_counts()
    r_ideal = recover_from_counts(ideal, n) == s
    r_noisy = recover_from_counts(noisy, n) == s
    print(f"{n}  {2 * n:>6}  {compiled.depth():>5}
{compiled.size():>5}",
          f"  {str(r_ideal):>14}  {str(r_noisy):>14}")
```

Output

```
n  qubits  depth  gates  ideal recovery  noisy recovery
2       4      5      8            True            True
3       6      5     12            True            True
4       8      6     17            True            True
5      10      6     21            True            True
6      12      7     26            True            True
```

```
# Do more shots help under noise? (n = 6)
s = "110101"
qc = transpile(simon_circuit_for(s), noisy_backend)
for shots in [64, 256, 1024, 4096]:
    noisy = noisy_backend.run(qc, shots=shots).result().get_counts()
    valid = sum(c for y, c in noisy.items() if dot(y, s) == 0)
```

```
    print(f"shots = {shots:>4}: valid fraction {valid / shots:.3f},",
          "recovered:", recover_from_counts(noisy, 6) == s)
```

Output

```
shots =   64: valid fraction 0.844, recovered: True
shots =  256: valid fraction 0.887, recovered: True
shots = 1024: valid fraction 0.901, recovered: True
shots = 4096: valid fraction 0.884, recovered: True
```

Observation

Every ideal run returns only strings with $y \cdot s = 0$ and the hidden string is recovered for all n. Under noise, a growing fraction of shots lands on strings that violate the condition, because the depth and the number of noisy CX gates increase with n. Ranking the outcomes by frequency and taking equations from the top still recovered the string in the sizes tested, since the valid strings remain the most frequent ones. **More shots sharpen the frequency ranking but do not change the fraction of valid outcomes**, so they help recovery only up to the point where noise makes an invalid string as frequent as a valid one.

viii. Conclusion

Simon’s algorithm turned a hidden XOR symmetry into a set of linear equations over GF(2): every measured string was orthogonal to the hidden string, n-1 independent strings fixed it uniquely, and direct oracle queries confirmed the collision structure f(x) = f(x XOR s). Against a classical collision search the number of oracle calls grew only linearly with n instead of exponentially, which is why such periodicities matter for cryptographic design. Noise introduced invalid outcomes that increased with circuit depth, but frequency ranking still recovered the string for up to six input bits.

CHAPTER 5

Experiment 5: Grover's Search Algorithm

Implementation and Programming of Grover's Search Algorithm Using Qiskit

i. Aim

To implement Grover's search algorithm using Qiskit and investigate its application for searching an unstructured database using quantum amplitude amplification.

ii. Setup

The notebook installs Qiskit and Qiskit Aer, then imports the simulator, visualisation and helper functions used by every task.

```
%pip install --quiet qiskit qiskit-aer matplotlib pylatexenc
```

```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

from qiskit import QuantumCircuit, transpile
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error

backend = AerSimulator()
```

iii. Task 1: Basic Grover Search

Problem Statement

The search space contains the four states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ and exactly one of them, $|11\rangle$, is the target. Construct the Grover circuit in the stages Hadamard, oracle, diffusion and measurement, record the histogram, the most probable state and the search success probability, and explain amplitude amplification.

Qiskit Code and Output

```
qc = QuantumCircuit(2, 2)

# Create uniform superposition
qc.h([0, 1])

# Oracle: mark |11>
qc.cz(0, 1)

# Diffusion operator
qc.h([0, 1])
qc.x([0, 1])
qc.h(1)
qc.cx(0, 1)
qc.h(1)
qc.x([0, 1])
qc.h([0, 1])

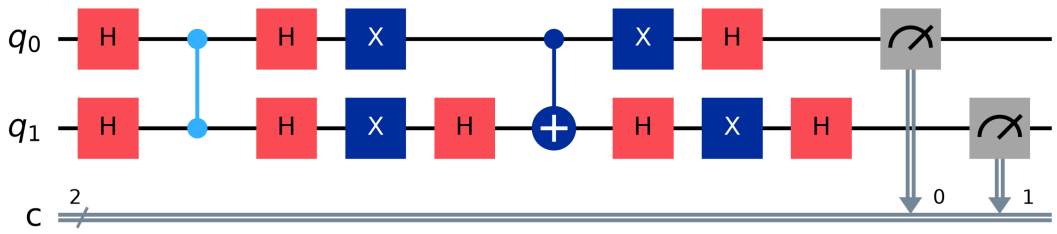
# Measurement
qc.measure([0, 1], [0, 1])

print(qc.draw())
display(qc.draw("mpl"))

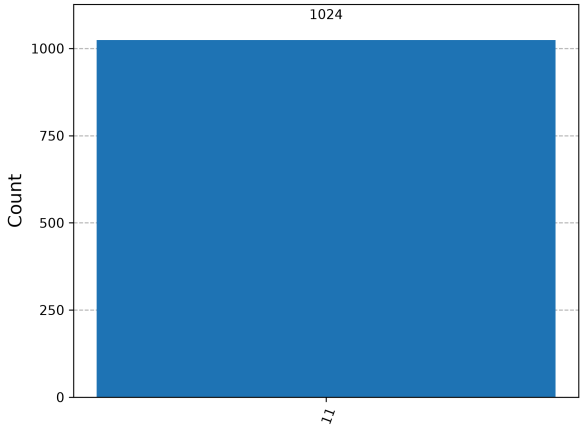
result = backend.run(
    transpile(qc, backend), shots=1024
).result()
counts = result.get_counts()
print(counts)
print("Most probable state:", max(counts, key=counts.get))
print("Success probability:", counts.get("11", 0) / 1024)
display(plot_histogram(counts))
```

Output

```
q_0: ┤H├──■──┤H├┤X├──────■──┤X├┤H├──────┤M├──
q_1: ┤H├──■──┤H├┤X├┤H├┤X├──┤H├┤X├┤H├┤X├┤H├┤M├
c: 2/═══════════════════════════════════════╩═╩═
                                           0 1
```




```
{'11': 1024}
Most probable state: 11
Success probability: 1.0
```




```
# Statevector verification after each stage
stage = QuantumCircuit(2)
stage.h([0, 1])
print("After Hadamard:", np.round(Statevector(stage).data, 3))
stage.cz(0, 1)
print("After oracle:  ", np.round(Statevector(stage).data, 3))
stage.h([0, 1]); stage.x([0, 1]); stage.h(1)
stage.cx(0, 1); stage.h(1); stage.x([0, 1]); stage.h([0, 1])
print("After diffusion:", np.round(Statevector(stage).data, 3))
```

Output

```
After Hadamard: [0.5+0.j 0.5+0.j 0.5+0.j 0.5+0.j]
After oracle:   [ 0.5+0.j  0.5+0.j  0.5+0.j -0.5+0.j]
After diffusion: [ 0.+0.j  0.+0.j  0.+0.j -1.+0.j]
```

Observation

The measurement gives **11 on all 1024 shots**, a success probability of 1. The statevector shows how: the Hadamards put equal amplitude 0.5 on every state, the oracle flips the sign of the $|11\rangle$ amplitude, and the diffusion

operator reflects every amplitude about the mean, which drives the three unmarked amplitudes to 0 and the marked one to 1. For $N = 4$ and one marked state a **single Grover iteration is exact**.

iv. Task 2: Oracle Design for a Real-World Search Problem

Problem Statement

Four records are represented by $00, 01, 10, 11$ and the record $|10\rangle$ satisfies the search condition. Design an oracle that marks only this state, run Grover search with it, and repeat the experiment for every target $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. States are written in Qiskit order, with qubit 1 on the left.

Qiskit Code and Output

```
oracle = QuantumCircuit(2)
# Mark |10>: flip qubit 0 so that CZ acts on the target
oracle.x(0)
oracle.cz(0, 1)
oracle.x(0)
print(oracle.draw())
```

Output

```
q_0: ┤ X ├──■──┤ X ├
q_1: ──────■──────
```

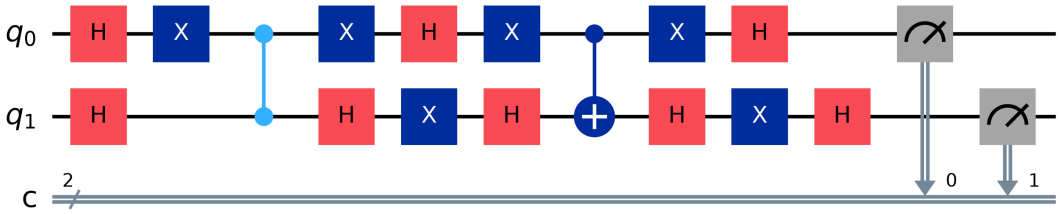
```
def grover_2q(target):
    qc = QuantumCircuit(2, 2)
    # Superposition
    qc.h([0, 1])
    # Oracle: X on the qubits that are 0 in the target, then CZ
    zeros = [i for i, bit in enumerate(reversed(target)) if bit
== "0"]
    if zeros:
        qc.x(zeros)
    qc.cz(0, 1)
    if zeros:
        qc.x(zeros)
    # Diffusion
    qc.h([0, 1])
    qc.x([0, 1])
    qc.h(1)
    qc.cx(0, 1)
    qc.h(1)
    qc.x([0, 1])
    qc.h([0, 1])
    qc.measure([0, 1], [0, 1])
    return qc

qc = grover_2q("10")
print(qc.draw())
display(qc.draw("mpl"))
```

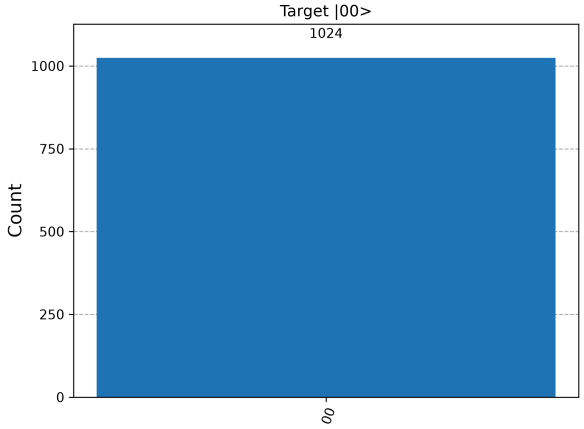
```
print("target  most probable  correct?")
for target in ["00", "01", "10", "11"]:
    counts = backend.run(
        transpile(grover_2q(target), backend), shots=1024
    ).result().get_counts()
    best = max(counts, key=counts.get)
    print(f"{target:>6}  {best:>13}  {best == target}")
    display(plot_histogram(counts, title=f"Target |{target}>"))
```

Output

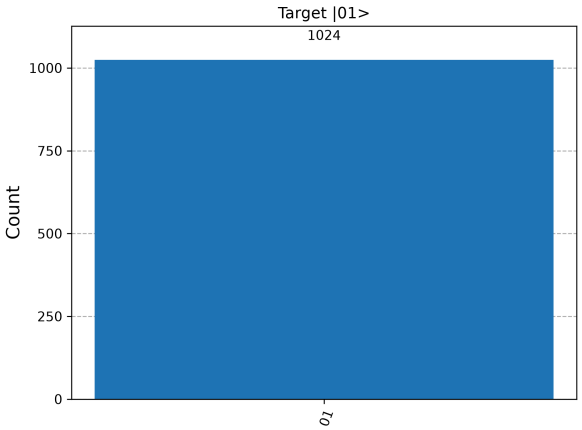
```
q_0: ┤H├┤X├┤X├┤H├┤X├┤X├┤H├┤M├
q_1: ┤H├┤H├┤X├┤H├┤X├┤H├┤X├┤M├
c: 2/═══════════════════════╩═╩
                           0 1
```



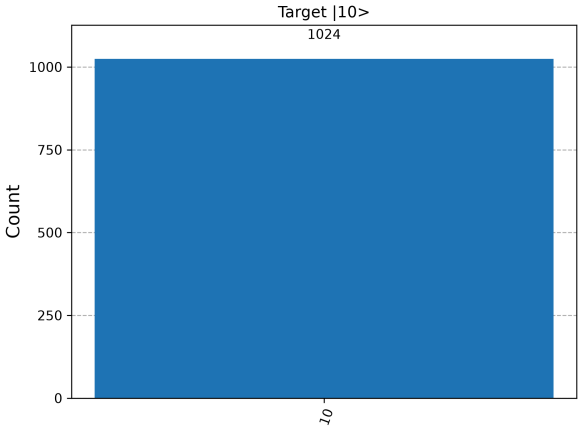

```
target  most probable  correct?
    00             00  True
```



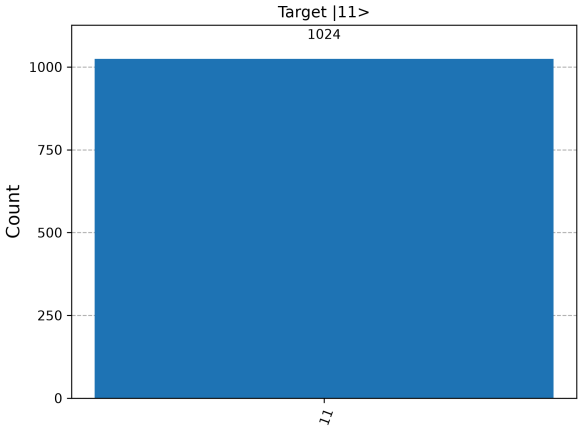

```
    01             01  True
```




```
10              10  True
```




```
11              11  True
```



Observation

Grover search for each target record

Target	Most probable state	Correct?
$\|00\rangle$	00	Yes
$\|01\rangle$	01	Yes
$\|10\rangle$	10	Yes
$\|11\rangle$	11	Yes

Each target was found with **probability 1** after one iteration. **Only the oracle changed between the four runs**: X gates surround the CZ so that it flips the phase of the wanted record, while the diffusion operator stayed identical. The oracle carries the problem-specific information and the rest of Grover's circuit is generic.

v. Task 3: Optimal Number of Grover Iterations

Problem Statement

The probability of finding the target does not increase indefinitely with repeated Grover iterations. For $N = 4, 8, 16$ (2, 3 and 4 qubits) apply the Grover iteration different numbers of times, measure the target probability, compare the experimental best with the estimate $\lfloor \frac{\pi}{4}\sqrt{\frac{N}{M}} \rfloor$, and explain why more iterations eventually reduce the probability.

Qiskit Code and Output

```
# Basic iteration estimate
N = 16
M = 1
iterations = int(np.floor((np.pi / 4) * np.sqrt(N / M)))
print("Recommended iterations:", iterations)
```

Output

```
Recommended iterations: 3
```

```
# n-qubit Grover circuit with a list of marked states
def phase_flip(qc, n, target):
    zeros = [i for i, bit in enumerate(reversed(target)) if bit
== "0"]
    if zeros:
        qc.x(zeros)
    qc.h(n - 1)
    qc.mcx(list(range(n - 1)), n - 1)
    qc.h(n - 1)
    if zeros:
        qc.x(zeros)

def grover(n, targets, iterations):
    qc = QuantumCircuit(n, n)
    qc.h(range(n))
    for _ in range(iterations):
        # Oracle
        for target in targets:
            phase_flip(qc, n, target)
        # Diffusion
        qc.h(range(n))
        phase_flip(qc, n, "0" * n)
        qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

def success(n, targets, iterations, shots=1024):
    counts = backend.run(
        transpile(grover(n, targets, iterations), backend),
        shots=shots
```

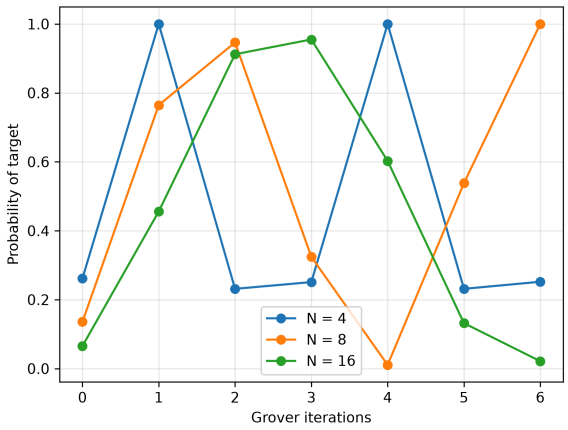
```
    ).result().get_counts()
    return sum(counts.get(t, 0) for t in targets) / shots

results = {}
for n in [2, 3, 4]:
    target = "1" * n
    probs = [success(n, [target], k) for k in range(7)]
    results[n] = probs
    # first peak: the iteration after which the probability drops
    best = next(k for k in range(6) if probs[k] >= probs[k + 1])
    rec = int(np.floor((np.pi / 4) * np.sqrt(2**n)))
    print(f"n = {n}, N = {2**n}: recommended {rec}, experimental
best {best}")
    print("   P(target) per iteration:", [round(p, 3) for p in probs])

for n, probs in results.items():
    plt.plot(range(7), probs, marker="o", label=f"N = {2**n}")
plt.xlabel("Grover iterations")
plt.ylabel("Probability of target")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Output

```
n = 2, N = 4: recommended 1, experimental best 1
   P(target) per iteration: [0.262, 1.0, 0.231, 0.251, 1.0, 0.231,
0.252]
n = 3, N = 8: recommended 2, experimental best 2
   P(target) per iteration: [0.136, 0.765, 0.946, 0.324, 0.011, 0.538,
1.0]
n = 4, N = 16: recommended 3, experimental best 3
   P(target) per iteration: [0.065, 0.456, 0.912, 0.955, 0.603, 0.132,
0.021]
```



Observation

Optimal number of Grover iterations

Qubits	Search space N	Recommended iterations	Experimental best
2	4	1	1
3	8	2	2
4	16	3	3

The first experimental peak agreed with the estimate $\lfloor \frac{\pi}{4}\sqrt{N} \rfloor$ in all three cases, and the plot shows the probability **rising, peaking and then falling again**, with a second peak appearing later because the rotation is periodic. Each iteration rotates the state by a fixed angle 2θ with $\sin\theta = \sqrt{\frac{M}{N}}$ in the plane spanned by the marked and unmarked states. The probability is $\sin^2((2k+1)\theta)$, so **after the peak the rotation overshoots the target** and the amplitude moves back toward the unmarked states. More iterations do not mean a better search; the count must be chosen from N and M.

vi. Task 4: Multiple Target States

Problem Statement

Modify Grover's algorithm so that more than one state is marked. In the four-state space mark $|01\rangle$ and $|10\rangle$, construct the oracle, record the histogram and the probability of each marked state, compare $M = 1$ with $M = 2$ and investigate how the required number of iterations changes.

Qiskit Code and Output

```
qc = QuantumCircuit(2, 2)

# Superposition
qc.h([0, 1])

# Mark |01>
qc.x(1)
qc.cz(0, 1)
qc.x(1)

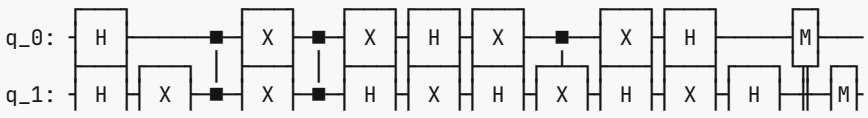
# Mark |10>
qc.x(0)
qc.cz(0, 1)
qc.x(0)

# Diffusion
qc.h([0, 1])
qc.x([0, 1])
qc.h(1)
qc.cx(0, 1)
qc.h(1)
qc.x([0, 1])
qc.h([0, 1])

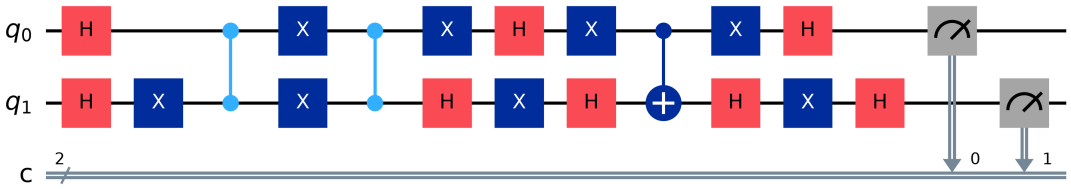
qc.measure([0, 1], [0, 1])
print(qc.draw())
display(qc.draw("mpl"))

counts = backend.run(
    transpile(qc, backend), shots=1024
).result().get_counts()
print(counts)
for state in ["01", "10"]:
    print(f"P({state}) =", round(counts.get(state, 0) / 1024, 3))
display(plot_histogram(counts))
```

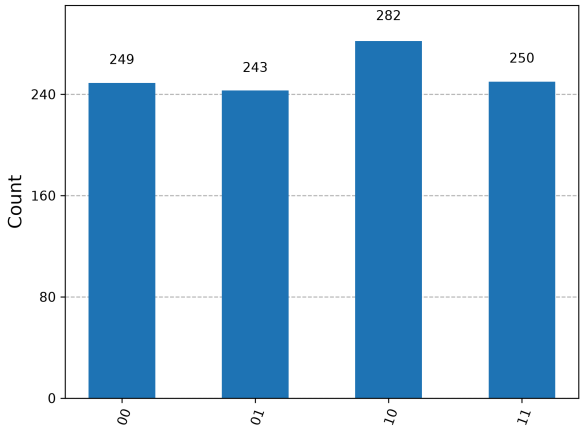
Output








```
{'10': 282, '01': 243, '00': 249, '11': 250}
P(01) = 0.237
P(10) = 0.275
```




```
# Iterations needed for M = 1 and M = 2 in N = 4 and N = 8
for n in [2, 3]:
    N = 2**n
    for targets in [["1" * n], ["0" + "1" * (n - 1), "1" + "0" * (n
- 1)]]:
        M = len(targets)
        rec = int(np.floor((np.pi / 4) * np.sqrt(N / M)))
        probs = [round(success(n, targets, k), 3) for k in range(4)]
        print(f"N = {N}, M = {M}: recommended {rec} iteration(s),",
              "P(marked) for k = 0..3:", probs)
```

Output

```
N = 4, M = 1: recommended 1 iteration(s), P(marked) for k = 0..3:
[0.253, 1.0, 0.243, 0.252]
N = 4, M = 2: recommended 1 iteration(s), P(marked) for k = 0..3:
[0.497, 0.517, 0.483,
    0.503]
N = 8, M = 1: recommended 2 iteration(s), P(marked) for k = 0..3:
[0.119, 0.797, 0.951,
```

```
    0.317]
N = 8, M = 2: recommended 1 iteration(s), P(marked) for k = 0..3:
[0.265, 1.0, 0.251, 0.272]
```

Observation

With two marked states in $N = 4$ the histogram after one iteration is **flat: every state has probability close to 0.25**, so each marked state keeps 0.25 and the search gains nothing. This is the special case $M = \frac{N}{2}$, where $\theta = \frac{\pi}{4}$ and one iteration rotates the state straight through the marked subspace and out again; the recommended count $\lfloor \frac{\pi}{4}\sqrt{2} \rfloor = 1$ is simply not useful here.

In $N = 8$ the effect of M is visible: one marked state needs **2 iterations** (probability about 0.95), while two marked states reach **probability 1 after a single iteration**. **More marked states need fewer iterations** because the rotation angle per iteration, 2θ with $\sin\theta = \sqrt{\frac{M}{N}}$, grows with M, so the state reaches the marked subspace sooner.

vii. Task 5: Noise, Search Accuracy and Scalability

Problem Statement

Investigate how noise influences Grover search as the number of qubits grows. For $n = 2, 3, 4, 5$ run the optimal-iteration circuit on an ideal simulator and on a noisy simulator (depolarizing error of 1 percent on single-qubit gates and 3 percent on CX), and record qubits, N, depth, gate count and success probability. Explain why a theoretically faster search becomes difficult on noisy hardware.

Qiskit Code and Output

```
# Noise model
noise_model = NoiseModel()
error_1q = depolarizing_error(0.01, 1)
error_2q = depolarizing_error(0.03, 2)
noise_model.add_all_qubit_quantum_error(error_1q, ["h", "x", "sx"])
noise_model.add_all_qubit_quantum_error(error_2q, ["cx"])

ideal_backend = AerSimulator()
noisy_backend = AerSimulator(noise_model=noise_model)

print("n   N  iters  depth  gates  ideal P  noisy P")
ideal_probs, noisy_probs = [], []
for n in [2, 3, 4, 5]:
    N = 2**n
    target = "1" * n
    k = int(np.floor((np.pi / 4) * np.sqrt(N)))
    qc = grover(n, [target], k)
    compiled = transpile(qc, noisy_backend)

    ideal = ideal_backend.run(
        transpile(qc, ideal_backend), shots=1024
    ).result().get_counts()
    noisy = noisy_backend.run(compiled,
shots=1024).result().get_counts()

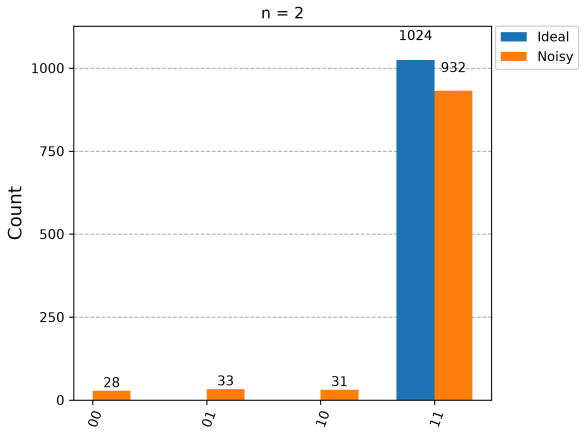
    p_ideal = ideal.get(target, 0) / 1024
    p_noisy = noisy.get(target, 0) / 1024
    ideal_probs.append(p_ideal)
    noisy_probs.append(p_noisy)
    print(f"{n}  {N:>2}  {k:>5}  {compiled.depth():>5}
{compiled.size():>5}",
          f"  {p_ideal:>7.3f}  {p_noisy:>7.3f}")
    if n <= 3:
        display(plot_histogram([ideal, noisy], legend=["Ideal",
"Noisy"],
                               title=f"n = {n}"))

plt.plot([2, 3, 4, 5], ideal_probs, marker="o", label="Ideal")
plt.plot([2, 3, 4, 5], noisy_probs, marker="o", label="Noisy")
plt.xlabel("Qubits")
```

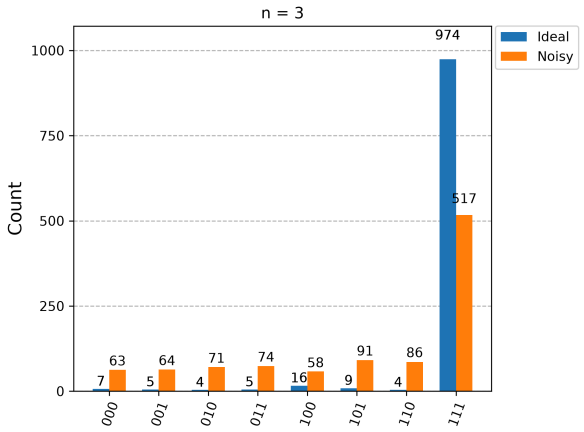
```
plt.ylabel("Success probability")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Output

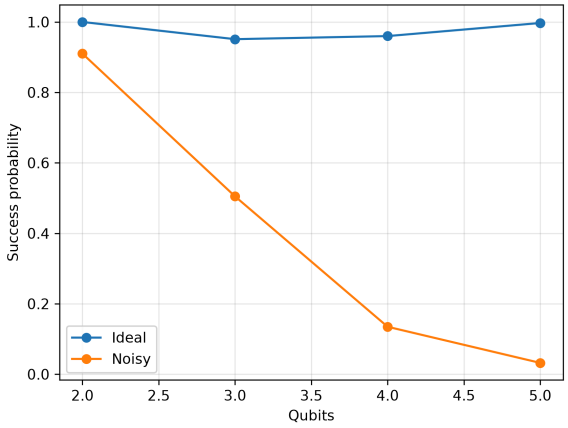
```
n   N  iters  depth  gates  ideal P  noisy P
2   4      1      8     12    1.000    0.910
```




```
3   8      2     50     82     0.951    0.505
```




```
4  16      3    170    230     0.960    0.135
5  32      4    603    759     0.997    0.032
```



Observation

Ideal success stays near 1 for every size (it dips slightly for $N = 8$ and $N = 32$ because the integer iteration count cannot land exactly on the peak). Under noise the success probability **falls steadily with the number of qubits**: the multi-controlled gates in the oracle and diffusion decompose into many CX gates, so depth and gate count grow quickly and each extra iteration multiplies the chance of an error. For 5 qubits the noisy circuit is already far from the ideal result.

This is why Grover's quadratic speedup is hard to use on NISQ hardware: the advantage needs $\sqrt{N}$ coherent iterations of a deep circuit, but **depolarizing noise washes out the amplified amplitude faster than the iterations build it up**, and without error correction the correct state loses its lead to the uniform background.

viii. Conclusion

- **Role of the oracle**: it marks the solution by flipping its phase and holds all the problem-specific information.
- **Diffusion operator**: it reflects every amplitude about the mean, converting the phase mark into a larger probability.
- **Initial Hadamards**: they create the uniform superposition in which every candidate is queried at once.
- **Quadratic, not exponential, speedup**: each iteration rotates the state by a fixed small angle, so about $\sqrt{N}$ rotations are needed; this is provably optimal for unstructured search.
- **Marked states and iterations**: more marked states enlarge the rotation angle and reduce the optimal count.
- **Over-iteration**: the rotation overshoots the target and the probability falls again.
- **Noise**: it shrinks the amplified amplitude every iteration and the success probability drops with circuit size.
- **NISQ limits**: deep multi-controlled circuits and the lack of error correction make large searches impractical today.

Grover's algorithm found the target in every noise-free experiment, the optimal iteration count matched the theoretical estimate for N = 4, 8 and 16, the multi-target case showed why the count depends on M, and the noisy runs showed the practical ceiling set by circuit depth.

CHAPTER 6

Experiment 6: Shor's Algorithm

Implementation and Investigation of Shor's Algorithm Using Qiskit

i. Aim

To implement and investigate Shor's algorithm using Qiskit for integer factorization, and to study period finding, quantum interference, classical post-processing and the effect of noise on quantum computation.

ii. Setup

The notebook installs Qiskit and Qiskit Aer, then imports the simulator, visualisation and helper functions used by every task.

```
%pip install --quiet qiskit qiskit-aer matplotlib pylatexenc
```

```
from math import gcd
from fractions import Fraction

import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

from qiskit import (
    QuantumCircuit,
    QuantumRegister,
    ClassicalRegister,
    transpile,
)
from qiskit.circuit.library import QFTGate
from qiskit.quantum_info import Operator
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error

simulator = AerSimulator()
```

iii. Task 1: Classical Period Finding

Problem Statement

For $N = 15$ and $a = 7$, calculate $a^x \bmod N$ for $x = 0, 1, ..., 7$ and determine the smallest positive integer r with $a^r \bmod N = 1$. Verify that $7^r \bmod 15 = 1$.

Qiskit Code and Output

```
N = 15
a = 7

print("N =", N)
print("a =", a)
print("gcd(a, N) =", gcd(a, N))

if gcd(a, N) != 1:
    raise ValueError("Choose a coprime base.")

print("\nModular exponentiation sequence:")
for x in range(8):
    value = pow(a, x, N)
    print(f"x = {x}, a^x mod N = {value}")

# Find the order classically
r = 1
while pow(a, r, N) != 1:
    r += 1
print("\nPeriod r =", r)
print("Verification: 7^r mod 15 =", pow(a, r, N))
```

Output

```
N = 15
a = 7
gcd(a, N) = 1

Modular exponentiation sequence:
x = 0, a^x mod N = 1
x = 1, a^x mod N = 7
x = 2, a^x mod N = 4
x = 3, a^x mod N = 13
x = 4, a^x mod N = 1
x = 5, a^x mod N = 7
x = 6, a^x mod N = 4
x = 7, a^x mod N = 13

Period r = 4
Verification: 7^r mod 15 = 1
```

Observation

Modular exponentiation table for a = 7, N = 15

x	0	1	2	3	4	5	6	7
$7^x \bmod 15$	1	7	4	13	1	7	4	13

The sequence repeats as $1, 7, 4, 13$, so the **period is $r = 4$** and $7^4 \bmod 15 = 1$ confirms it. Because r is even and $7^2 = 4 \neq -1 \pmod{15}$, the factors follow from $\gcd(7^2 \pm 1, 15) = 3, 5$.

iv. Task 2: Construction of Modular Multiplication Gates

Problem Statement

For $N = 15$ construct the reversible operator $U_b|x\rangle = |bx \bmod 15\rangle$ for $0 \leq x < 15$, leaving the unused basis state $|15\rangle$ unchanged. Verify the modular powers $7^1, 7^2, 7^4 \bmod 15$ and explain why the operator must be reversible.

Qiskit Code and Output

```
def modular_multiplication_gate(b, N=15):
    n_target = (N - 1).bit_length()
    dim = 2**n_target

    if gcd(b, N) != 1:
        raise ValueError("b and N must be coprime.")

    U = np.zeros((dim, dim), dtype=complex)

    for x in range(dim):
        if x < N:
            y = (b * x) % N
        else:
            y = x
        U[y, x] = 1

    return Operator(U).to_instruction()

U7 = modular_multiplication_gate(7, 15)
U4 = modular_multiplication_gate(4, 15)

print("Multiplication gate for b = 7 constructed.")
print("Multiplication gate for b = 4 constructed.")

print("7^1 mod 15 =", pow(7, 1, 15))
print("7^2 mod 15 =", pow(7, 2, 15))
print("7^4 mod 15 =", pow(7, 4, 15))
```

Output

```
Multiplication gate for b = 7 constructed.
Multiplication gate for b = 4 constructed.
7^1 mod 15 = 7
7^2 mod 15 = 4
7^4 mod 15 = 1
```

```
# Check that U7 is a permutation matrix and therefore reversible
U = Operator(U7).data.real
print("Non-zero entries per row:   ", U.sum(axis=1).astype(int))
print("Non-zero entries per column:", U.sum(axis=0).astype(int))
print("Unitary (U U^dagger = I):", np.allclose(U @ U.T, np.eye(16)))
print("U7 maps |1> to |", int(np.argmax(U[:, 1])), ">")
print("U7 maps |7> to |", int(np.argmax(U[:, 7])), ">")
```

Output

```
Non-zero entries per row:    [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
Non-zero entries per column: [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
Unitary (U U^dagger = I): True
U7 maps |1> to | 7 >
U7 maps |7> to | 4 >
```

Observation

Each multiplication operator is a **permutation matrix** with exactly one non-zero entry per row and per column, and $UU^\dagger = I$ holds. U_7 sends $|1\rangle$ to $|7\rangle$ and $|7\rangle$ to $|4\rangle$, matching $7^1 = 7$ and $7^2 = 4 \bmod 15$. **The operator must be reversible** because every quantum gate is unitary: multiplication by b coprime to N is a bijection on $\{0, ..., 14\}$, so no information is lost and the inverse (multiplication by b^{-1}) exists. A non-invertible map could not be a gate.

v. Task 3: Quantum Circuit Construction and Simulation

Problem Statement

Construct and simulate a period-finding circuit for $N = 15$, $a = 7$ with a 4-qubit control register in superposition, a target register initialised to $|1\rangle$, controlled modular multiplication gates $U_{a^{2^k}}$, the inverse QFT and measurement of the control register. Record the circuit, the counts and the histogram, and explain the two registers and the inverse QFT.

Qiskit Code and Output

```
N = 15
a = 7
n_control = 4
n_target = (N - 1).bit_length()

def build_shor_circuit(N, a, n_control=4):
    n_target = (N - 1).bit_length()

    control = QuantumRegister(n_control, "c")
    target = QuantumRegister(n_target, "t")
    measured = ClassicalRegister(n_control, "m")

    qc = QuantumCircuit(control, target, measured)

    # Prepare target register in |1>
    qc.x(target[0])

    # Prepare control register in superposition
    qc.h(control)

    # Controlled modular exponentiation
    for k in range(n_control):
        b = pow(a, 2**k, N)
        gate = modular_multiplication_gate(b, N)
        controlled_gate = gate.control()
        qc.append(controlled_gate, [control[k], *target])

    # Inverse QFT
    qc.append(QFTGate(n_control).inverse(), list(control))

    # Measure control register
    qc.measure(control, measured)

    return qc

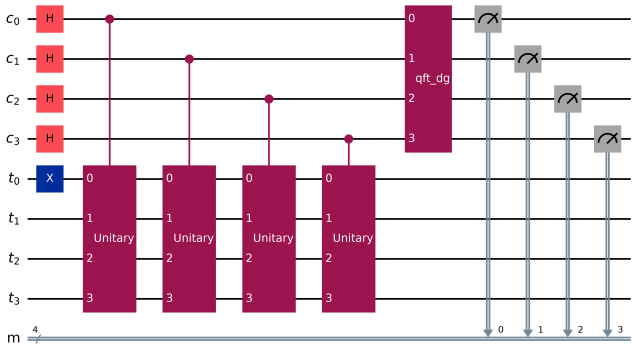
qc = build_shor_circuit(N, a, n_control)
display(qc.draw("mpl"))

compiled = transpile(qc, simulator)
```

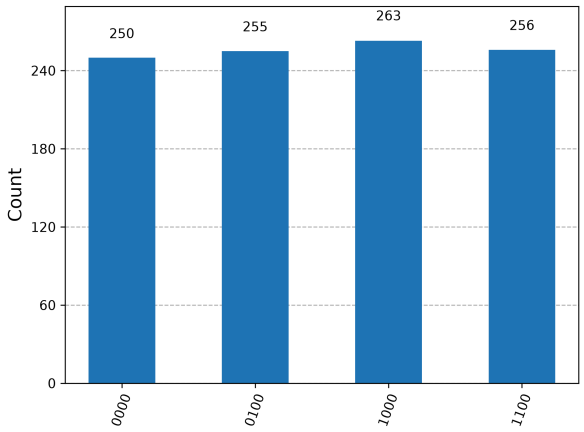
```
result = simulator.run(compiled, shots=1024).result()
counts = result.get_counts()

print("\nMeasurement results:")
print(counts)
for bitstring in sorted(counts):
    m = int(bitstring, 2)
    print(f"m = {m:>2} ({bitstring}): {counts[bitstring]} shots,",
          f"phase = {m}/16 = {Fraction(m, 16)}")
display(plot_histogram(counts))
```

Output




```
Measurement results:
{'0100': 255, '1100': 256, '0000': 250, '1000': 263}
m =  0 (0000): 250 shots, phase = 0/16 = 0
m =  4 (0100): 255 shots, phase = 4/16 = 1/4
m =  8 (1000): 263 shots, phase = 8/16 = 1/2
m = 12 (1100): 256 shots, phase = 12/16 = 3/4
```



Observation

The measurement outcomes concentrate on $\boldsymbol{m = 0, 4, 8, 12}$, each with about a quarter of the shots. These are the 4-bit representations of the phases 0, $\frac{1}{4}$, $\frac{1}{2}$, $\frac{3}{4}$, whose denominators point to the period $r = 4$.

- **Control register**: holds the superposition of all exponents x; after the controlled multiplications it is entangled with the value $a^x \bmod N$ in the target register, so it carries the periodicity.
- **Target register**: starts in $|1\rangle$ and accumulates $a^x \bmod N$ through the controlled gates $U_{a^{2^k}}$, one for each control qubit k.
- **Inverse QFT**: converts the periodic pattern in the control register into peaks at multiples of $\frac{2^n}{r}$, which is what the measurement reads out as m.

vi. Task 4: Period Estimation and Factor Recovery

Problem Statement

For each measured integer m estimate the phase $\varphi = \frac{m}{2^n}$, where n is the number of control qubits, use continued fractions to obtain a candidate denominator, test its multiples to find a valid order (the denominator may be only a divisor of the true period) and, once an even period r is found, calculate $p = \gcd\left(a^{\frac{r}{2}} - 1, N\right)$ and $q = \gcd\left(a^{\frac{r}{2}} + 1, N\right)$ and verify the factorisation.

Qiskit Code and Output

```
def estimate_period(m, n_control, a, N):
    if m == 0:
        return None

    phase = Fraction(m, 2**n_control)
    approximation = phase.limit_denominator(N)
    denominator = approximation.denominator

    # Test multiples because the denominator can
    # be a divisor of the true order.
    for candidate in range(denominator, N + 1, denominator):
        if pow(a, candidate, N) == 1:
            return candidate

    return None

print("Period estimates from measurements:")

periods = []

for bitstring, frequency in counts.items():
    m = int(bitstring, 2)
    r_candidate = estimate_period(m, n_control, a, N)
    print(
        f"Outcome={bitstring}, m={m}, "
        f"phase={Fraction(m, 2**n_control)}, "
        f"frequency={frequency}, "
        f"candidate period={r_candidate}"
    )
    if r_candidate is not None:
        periods.append(r_candidate)

# Try candidate periods and recover factors
factors = set()

for r_candidate in sorted(set(periods)):
    if r_candidate % 2 != 0:
        continue

    x = pow(a, r_candidate // 2, N)
```

```
    if x == 1 or x == N - 1:
        continue

    p = gcd(x - 1, N)
    q = gcd(x + 1, N)
    print(f"r = {r_candidate}: a^(r/2) mod N = {x},",
          f"gcd({x} - 1, {N}) = {p}, gcd({x} + 1, {N}) = {q}")

    if 1 < p < N and 1 < q < N:
        factors.add(p)
        factors.add(q)

print("\nNon-trivial factors:", sorted(factors))

if factors:
    p = min(factors)
    q = N // p
    print(f"Factorization: {N} = {p} x {q}")
    print("Verification:", p * q == N)
else:
    print("No factors recovered. Repeat the experiment.")
```

Output

```
Period estimates from measurements:
Outcome=0100, m=4, phase=1/4, frequency=255, candidate period=4
Outcome=1100, m=12, phase=3/4, frequency=256, candidate period=4
Outcome=0000, m=0, phase=0, frequency=250, candidate period=None
Outcome=1000, m=8, phase=1/2, frequency=263, candidate period=4
r = 4: a^(r/2) mod N = 4, gcd(4 - 1, 15) = 3, gcd(4 + 1, 15) = 5

Non-trivial factors: [3, 5]
Factorization: 15 = 3 x 5
Verification: True
```

Observation

Period estimation from the measured phases

Outcome m	Phase $\frac{m}{16}$	Continued fraction	Candidate period
0	0	none	none (no information)
4	1/4	1/4	4
8	1/2	1/2	2, then 4 after testing multiples
12	3/4	3/4	4

Three of the four outcomes lead to the period $r = 4$; $m = 8$ first suggests the divisor 2, which fails the test $a^2 \bmod 15 = 4 \neq 1$, so its multiple 4 is taken. With $r = 4$, $a^{\frac{r}{2}} = 7^2 \bmod 15 = 4$ and the GCD step gives $\gcd(3, 15) = 3$ and $\gcd(5, 15) = 5$, so $\mathbf{15 = 3 \times 5}$ is recovered and verified. The outcome $m = 0$ carries no information, which is why single measurements cannot be relied on and candidates must be checked.

vii. Task 5: Classical and Quantum Comparison

Problem Statement

For $N = 15$ and $a = 7$ compare the classical sequential search for the order with the quantum period-finding simulation: record the modular-power checks on one side and the shots and circuit resources on the other, and discuss oracle calls, circuit resources and computational complexity.

Qiskit Code and Output

```
N = 15
a = 7

# Classical order search
classical_checks = 0
r_classical = 1

while True:
    classical_checks += 1
    if pow(a, r_classical, N) == 1:
        break
    r_classical += 1

print("Classical period:", r_classical)
print("Classical modular-power checks:", classical_checks)

# Quantum simulation query count
# Each circuit execution uses the oracle circuit.
quantum_shots = 1024

print("Quantum circuit shots:", quantum_shots)
print("Quantum circuit uses controlled modular arithmetic.")
print("Transpiled depth:", compiled.depth())
print("Transpiled gate count:", compiled.size())
print("Qubits:", qc.num_qubits)
```

Output

```
Classical period: 4
Classical modular-power checks: 4
Quantum circuit shots: 1024
Quantum circuit uses controlled modular arithmetic.
Transpiled depth: 910
Transpiled gate count: 1246
Qubits: 8
```

Observation

Classical and quantum period finding for N = 15, a = 7

Parameter	Classical	Quantum
Main method	sequential order search	quantum period estimation
Period information	direct modular powers	measurement phases $\frac{m}{2^n}$
Post-processing	order verification	continued fractions and GCD
Hardware	classical processor	quantum circuit or simulator
Observed cost	4 modular-power checks	1024 shots of a deep circuit

For $N = 15$ the classical search needs only **4 checks**, far less work than the quantum simulation with its thousands of transpiled gates. **This experiment does not demonstrate a quantum speedup**: the simulator runs on classical hardware and the instance is tiny. The quantum advantage is asymptotic, polynomial in the number of bits of N against super-polynomial for the best classical factoring methods, and only appears for large N on a fault-tolerant machine.

viii. Task 6: Noise and Scalability Investigation

Problem Statement

Part A: run the Task 3 circuit under depolarizing noise (1 percent on `sx`, 3 percent on `cx`) and compare the counts with the ideal run. **Part B**: vary the number of control qubits over $3, 4, 5, 6$ and record circuit depth, gate count, number of distinct outcomes and whether the period can still be recovered.

Qiskit Code and Output

```
# Part A: noisy simulation of the Task 3 circuit
noise_model = NoiseModel()

error_1q = depolarizing_error(0.01, 1)
error_2q = depolarizing_error(0.03, 2)

# Apply errors to transpiled basis gates
noise_model.add_all_qubit_quantum_error(error_1q, ["sx"])
noise_model.add_all_qubit_quantum_error(error_2q, ["cx"])

noisy_backend = AerSimulator(noise_model=noise_model)

noisy_compiled = transpile(qc, noisy_backend)

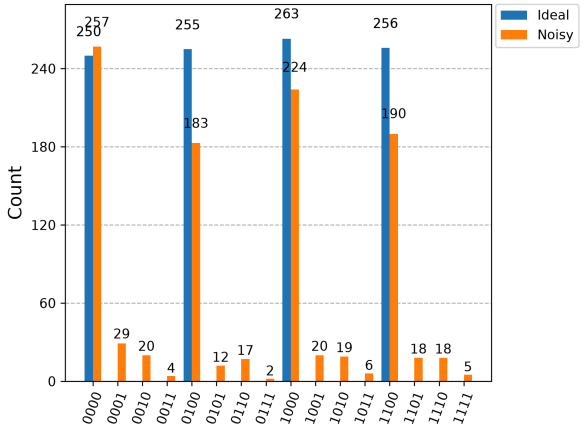
noisy_result = noisy_backend.run(
    noisy_compiled, shots=1024
).result()

noisy_counts = noisy_result.get_counts()

print("Noisy measurement results:")
print(noisy_counts)
ideal_peaks = sum(counts.get(k, 0) for k in ["0000", "0100", "1000",
"1100"])
noisy_peaks = sum(noisy_counts.get(k, 0) for k in ["0000", "0100",
"1000", "1100"])
print("Shots on m = 0, 4, 8, 12 (ideal):", ideal_peaks)
print("Shots on m = 0, 4, 8, 12 (noisy):", noisy_peaks)
display(plot_histogram([counts, noisy_counts], legend=["Ideal",
"Noisy"]))
```

Output

```
Noisy measurement results:
{'1000': 224, '0000': 257, '1100': 190, '0100': 183, '1001': 20,
'0001': 29, '1010': 19,
    '0101': 12, '0010': 20, '0110': 17, '1101': 18, '1110': 18,
'0111': 2, '1011': 6,
    '1111': 5, '0011': 4}
Shots on m = 0, 4, 8, 12 (ideal): 1024
Shots on m = 0, 4, 8, 12 (noisy): 854
```




```
# Part B: vary the number of control qubits
for n_control in [3, 4, 5, 6]:
    test_circuit = build_shor_circuit(N=15, a=7, n_control=n_control)

    compiled_test = transpile(test_circuit, simulator)

    result = simulator.run(compiled_test, shots=1024).result()
    counts_test = result.get_counts()

    periods_found = set()
    for bitstring in counts_test:
        r_found = estimate_period(int(bitstring, 2), n_control, 7, 15)
        if r_found is not None:
            periods_found.add(r_found)

    print("\nControl qubits:", n_control)
    print("Circuit depth:", compiled_test.depth())
    print("Total gates:", compiled_test.size())
    print("Distinct outcomes:", len(counts_test))
    print("Counts:", counts_test)
    print("Period recovered:", 4 in periods_found)
```

Output

```
Control qubits: 3
Circuit depth: 910
Total gates: 1240
Distinct outcomes: 4
Counts: {'100': 293, '000': 257, '110': 243, '010': 231}
Period recovered: True

Control qubits: 4
Circuit depth: 910
Total gates: 1246
Distinct outcomes: 4
Counts: {'1100': 252, '0100': 268, '0000': 252, '1000': 252}
Period recovered: True

Control qubits: 5
Circuit depth: 910
```

```
Total gates: 1253
Distinct outcomes: 4
Counts: {'11000': 256, '10000': 274, '01000': 256, '00000': 238}
Period recovered: True

Control qubits: 6
Circuit depth: 910
Total gates: 1261
Distinct outcomes: 4
Counts: {'110000': 245, '010000': 265, '000000': 241, '100000': 273}
Period recovered: True
```

Observation

Part A. Under noise the four ideal peaks $m = 0, 4, 8, 12$ remain the tallest bars, but a sizeable fraction of the shots spreads over the other twelve outcomes. The circuit is deep after transpilation, so even 1 to 3 percent depolarizing error per gate accumulates into a visible background; the period can still be read from the peaks, but a larger noise rate or deeper circuit would bury them.

Part B. More control qubits improve the phase resolution (2^n bins), so for $r = 4$ the peaks sit exactly on multiples of $\frac{2^n}{4}$ for every size tested, the number of distinct outcomes stays at four and the period is recovered each time. The transpiled depth is the same for all sizes because $7^{2^k} \bmod 15 = 1$ for $k \geq 2$, so the extra controlled multiplications are identities and only the QFT rotations add a few gates; for a general N each extra control qubit would add a full controlled modular multiplication. Classical simulation cost still grows exponentially with the total qubit count, which is why larger instances quickly become impractical to simulate. More shots sharpen the histogram but cannot remove the systematic effect of noise or the uninformative $m = 0$ outcome.

ix. Conclusion

Shor's algorithm was investigated for N = 15 and a = 7. The classical sequence 1, 7, 4, 13 gave the period 4; reversible modular multiplication gates were built as permutation matrices; the quantum period-finding circuit produced peaks at m = 0, 4, 8 and 12; continued fractions and the GCD step recovered the verified factorisation 15 = 3 x 5. The comparison with the classical search showed that a small simulation demonstrates the mechanism rather than a speedup, and the noise and scaling study showed how depolarizing errors spread the histogram and how resources grow with the control register.